Benemérita Universidad Autónoma de Chiapas

Por La Conciencia De La Necesidad De Servir

FACULTAD y LICENCIATURA:

Escuela de Tecnologías Digitales Aplicadas, Campus I

(Ingeniería en Desarrollo y Tecnologías de Software)

NOMBRE DE LAS MATERIAS:

Sistemas Digitales

Programación Avanzada

NOMBRE DE LA ACTIVIDAD:

Reporte de Avances del Proyecto Integrador

Sistema Universitario de Seguimiento y Análisis de Asistencia (SUSAN)

INTEGRANTES DEL EQUIPO:

Casanova Pérez Gabriela

Escobar Domínguez Josué Gabriel

Gómez Delgado Ades Yasir

González Guerrero Diego Enrique

Peña Morales José Manuel

Pérez Hernández Alan Eduardo

NOMBRE DE LOS CATEDRÁTICOS:

Dr. Daniel González Scarpulli

Dr. Yair González Baldizón

SEMESTRE: 3° **| GRADO:** “N”

FECHA Y LUGAR DE ENTREGA:

Octubre 30, 2026. Tuxtla Gutiérrez, Chiapas, México

ÍNDICE.

TEMA Y DESCRIPCIÓN. 4

PLANTEAMIENTO DEL PROBLEMA Y JUSTIFICACIÓN. 4

OBJETIVOS. 6

1. Objetivo general. 6

2. Objetivos específicos. 6

ALCANCE. 7

ARQUITECTURA GENERAL DEL SISTEMA. 8

1. Protocolo serie. 10

2. Secuencia de un registro por tarjeta. 11

3. Secuencia del pase de lista con cámara. 12

DISEÑO DEL CIRCUITO DE LA ESTACIÓN DE AULA. 13

1. Tarjeta propuesta: Arduino UNO R3. 13

2. Bloques del circuito. 13

3. Asignación de pines. 14

4. Esquema del circuito. 15

5. Lista de materiales. 17

CÓDIGO DE ARDUINO. 18

1. Archivos del diseño. 18

2. Protocolo serie. 19

3. Lógica del protocolo. 20

4. Programa principal. 21

5. Programa del puerto serie en el PC. 22

6. Estado de verificación. 24

DISEÑO Y CÓDIGO DE LOS FORMULARIOS. 25

1. Formularios del sistema. 25

2. Principios de diseño. 26

3. Diseño de las pantallas. 28

4. Organización del código. 36

5. Código de los formularios. 37

6. Estado de verificación. 41

ESTRUCTURA DE LA BASE DE DATOS. 42

1. Modelo entidad-relación. 43

2. Tablas. 49

3. Reglas de negocio en la base de datos. 51

4. Código de base de datos. 52

5. Estado de verificación. 57

AVANCE DEL PROYECTO. 58

1. Estado por parte......58
2. Integración de las materias......59
3. Próximos pasos......59
VIABILIDAD TÉCNICA Y RIESGOS......59
1. Materiales y software......60
2. Riesgos......61
REFERENCIAS BIBLIOGRÁFICAS......63

TEMA Y DESCRIPCIÓN.

Tema: Sistema Universitario de Seguimiento y Análisis de Asistencia (SUSAN): registro digital centralizado de asistencia académica, análisis del ausentismo mediante métodos numéricos y estación de aula con Arduino.

Descripción: SUSAN plantea la transición de un control de asistencia administrativo tradicional hacia un modelo estructurado y basado en datos. El docente abre la sesión de clase en una aplicación web; durante la ventana de tolerancia, una cámara del aula toma capturas periódicas, un servicio de reconocimiento compara cada rostro con las plantillas de los alumnos inscritos que otorgaron su consentimiento, y la base de datos consolida las lecturas en un solo registro de asistencia por alumno y sesión. El docente revisa los casos dudosos y puede corregir o justificar un registro. Con los historiales consolidados, un módulo de métodos numéricos calcula tendencias, proyecciones y un puntaje de riesgo por alumno para identificar patrones de ausentismo y orientar una intervención docente oportuna, sin automatizar las decisiones institucionales.

En este avance se agrega la **estación de aula**: un dispositivo con Arduino UNO R3, lector RFID MFRC522, pantalla LCD de 16x2 con interfaz I2C, tres LED, un zumbador y un pulsador, conectado por USB a la computadora del docente. Su propósito es registrar con una tarjeta RFID personal a los alumnos que no otorgaron su consentimiento para el reconocimiento facial, o a cualquier alumno cuando la cámara falla. SUSAN nunca reconoce un rostro sin consentimiento vigente; la tarjeta ofrece una vía de registro que no usa el rostro. La estación solo lee la tarjeta y muestra el resultado: nunca cierra una sesión, porque cerrarla exige la autenticación del docente en la aplicación web.

La división de trabajo es deliberada: la aplicación web, el servicio de reconocimiento y la base de datos controlan la información (alumnos, inscripciones, consentimientos, sesiones, reglas de negocio, historial y análisis numérico), y la estación de aula controla el contacto físico con el alumno en el escritorio (lectura de la tarjeta, aviso inmediato con pantalla, luces y sonido, y el enlace serie con la computadora). Una parte hace lo que la otra no debe hacer: el microcontrolador no guarda historiales ni decide quién puede registrarse, y la aplicación no puede leer una tarjeta ni dar aviso físico en el pupitre.

Estado de la estación: la estación se diseñó y se verificó solo en software; todavía no se ha probado con el hardware físico. La asociación entre el identificador (UID) de cada tarjeta y el alumno, así como su integración con las funciones de la base de datos, son pasos siguientes y no forman parte de lo ya hecho.

PLANTEAMIENTO DEL PROBLEMA Y JUSTIFICACIÓN.

En la Facultad de Contaduría y Administración Campus I de la Universidad Autónoma de Chiapas, la asistencia se registra con el pase de lista tradicional: al inicio de cada sesión el docente nombra a cada alumno y anota el resultado en una lista impresa, y al cierre del

periodo el conteo se vuelve a capturar a mano. Esa descripción y las cifras siguientes proceden del análisis del propio proyecto (primer avance, sección 1.1) y no de un dato institucional. De ahí se desprenden cuatro problemas:

1. **Tiempo de clase.** Con el supuesto de 5 minutos por sesión, 3 sesiones por semana y 16 semanas, el pase de lista consume 240 minutos, es decir, 4 horas de clase por grupo al semestre. Como **estimación**, un docente con cinco grupos destina unas 20 horas por semestre solo a nombrar alumnos. El ahorro real debe medirse en el aula; el proyecto no lo ha medido.
2. **Registro poco confiable.** El dato depende de que el alumno responda, lo que lo expone a la suplantación y a errores de captura, y la recaptura manual al cierre del periodo añade más errores.
3. **Sin historial consultable.** El alumno no sabe cuántas faltas acumula hasta que es tarde y los datos en papel no permiten reportes por materia, grupo o periodo sin trabajo manual. Con la regla del proyecto de que tres retardos equivalen a una ausencia y de que superar el 20 % de ausencias retira el derecho a examen, la falta de un acumulado a tiempo tiene consecuencias académicas. Como **estimación**, en un grupo de 48 sesiones el límite se supera en la décima falta equivalente (el 20 % de 48 es 9.6).
4. **Sin vía de registro para quien no consiente o cuando la cámara falla.** El reconocimiento facial trata el rostro como dato personal sensible y exige consentimiento firmado y vigente. Quien no lo otorga, o todo el grupo cuando la cámara no funciona, se registra hoy con el pase de lista manual del docente, que reintroduce el costo de tiempo y la dependencia de la respuesta oral que se quería evitar.

Las tres primeras comparten una causa: el dato se genera una sola vez, por una persona, en un medio que no puede consultarse ni verificarse. La cuarta es propia de este avance: ninguna vía única cubre a todos los alumnos sin renunciar al consentimiento.

Pregunta que guía el proyecto: ¿cómo diseñar un sistema que registre la asistencia en el aula con menor tiempo de clase, conserve un historial consolidado y consultable, analice numéricamente el ausentismo para orientar al docente a tiempo y, sin reconocer nunca un rostro sin consentimiento, ofrezca una vía alterna de registro con una tarjeta personal cuando el alumno no consintió o la cámara falla?

Justificación. El proyecto integra las dos materias y la base de datos de forma real: cada parte resuelve algo que las otras no pueden hacer bien.

- **La estación de aula (Sistemas Digitales)** responde en el momento y en el escritorio del aula: lee la tarjeta, muestra el resultado en la pantalla, los LED y el zumbador, y vigila el enlace con la computadora. La lectura, el aviso y el enlace serie ocurren en el equipo del docente, sin depender de la nube. Es además la vía de registro que no usa el rostro, y

por eso resuelve el cuarto problema. Si la computadora no responde a tiempo, la estación muestra el error y no confirma el registro.

- **El programa en Node y la aplicación web (Programación Avanzada)** manejan el puerto serie, validan lo que llega y presentan al docente la sesión, los casos dudosos y los reportes. Hacen lo que un microcontrolador no debe hacer: autenticar al docente, aplicar las reglas del negocio y cerrar la sesión.
- **La base de datos** guarda lo que un circuito no puede guardar: alumnos, inscripciones, consentimientos, sesiones, un solo registro de asistencia por alumno y sesión, y el historial completo con bitácora de auditoría. Sobre ese historial se calculan las tendencias y el riesgo por alumno que justifican el proyecto.
- **El servicio de reconocimiento** cubre el caso general sin intervención del alumno, que es lo que reduce el tiempo de pase de lista; la estación solo atiende a quien ese servicio no debe o no puede atender.

Ninguna parte basta sola: sin la estación, el alumno que no consiente depende del pase manual; sin la base de datos, la estación solo leería tarjetas que nadie puede asociar a un registro, historial ni análisis. Esta división también limita el riesgo: la estación no conserva historial ni decide si un alumno puede registrarse.

OBJETIVOS.

1. Objetivo general.

Diseñar e implementar un sistema centralizado de registro digital de asistencia académica y de análisis del ausentismo mediante métodos numéricos, que combine una aplicación web con formularios, una base de datos relacional, un servicio de reconocimiento facial con consentimiento y una estación de aula con Arduino conectada por puerto serie, para reducir el tiempo de pase de lista en el aula, generar historiales consolidados que permitan identificar patrones de ausentismo y orientar la intervención docente oportuna, y registrar por una vía alterna sin rostro a los alumnos que no otorgaron su consentimiento o cuando la cámara falla.

2. Objetivos específicos.

1. Desarrollar una aplicación web con formularios para la gestión académica (alumnos, docentes, materias, grupos, aulas, cámaras, horarios e inscripciones), la apertura y el cierre de la sesión, el pase de lista, la revisión de casos dudosos, los justificantes y los reportes, con acceso según el rol y sin que la interfaz contenga reglas de negocio.
2. Diseñar una base de datos relacional normalizada en PostgreSQL, con restricciones, disparadores y funciones que hagan cumplir las reglas de negocio del proyecto, un solo registro de asistencia por alumno y sesión, y una bitácora de auditoría, sin guardar vectores ni imágenes de rostros en las tablas.

3. Implementar un servicio de reconocimiento que vectorice los rostros capturados, los busque en un índice y atribuya la asistencia solo a alumnos inscritos con consentimiento vigente, de modo que quien no lo otorgue se registre por otra vía.
4. Aplicar métodos numéricos (regresión, suavizado, interpolación, proyección, búsqueda de raíces, control de cambios y puntaje de riesgo) a las series de asistencia para identificar tendencias, el cruce del límite de ausencias y los alumnos en riesgo.
5. Diseñar la estación de aula con Arduino UNO R3, lector RFID MFRC522, pantalla LCD 16x2, tres LED, zumbador y pulsador, que lea la tarjeta personal del alumno, dé aviso inmediato y nunca cierre una sesión.
6. Definir un protocolo serie de líneas ASCII (115200 baudios, 8N1) entre la estación y la computadora del docente, y desarrollar en Node el programa que maneje el puerto serie, valide cada línea recibida y responda con el resultado.
7. Validar cada módulo por separado (pruebas automatizadas de la base de datos, del servicio y del módulo numérico, pruebas de la lógica de la estación en la computadora y pruebas del programa del puerto con una conexión simulada) antes de integrarlos, y dejar documentado lo que aún no se ha probado con el hardware físico.

ALCANCE.

El sistema cubre el registro de asistencia de grupos de una institución con un docente por sesión, una cámara de aula (de red o la cámara del navegador) y una estación de aula conectada por USB a la computadora del docente. La aplicación web, la base de datos, el servicio de reconocimiento y el módulo numérico existen y se pueden demostrar en modo local con datos de prueba; la estación de aula está diseñada y verificada solo en software, y su integración con la base de datos queda como siguiente paso.

Incluye	No incluye
Aplicación web con nueve pantallas (P-01 a P-09) y acceso por rol (administrador, docente, alumno y técnico)	Aplicación móvil nativa; la interfaz es un portal web responsivo
Base de datos PostgreSQL con 20 tablas de datos (las 19 de la especificación y una de fotos de plantilla), disparadores y funciones	Calificaciones, kárdex u otro dato académico distinto de la asistencia
Servicio de reconocimiento con índice FAISS y consentimiento previo del alumno	Entrenamiento del modelo de visión; se usa uno preentrenado
Pase de lista manual de respaldo para quien no consiente o sin cámara activa	Detección de vida (anti-spoofing); riesgo abierto que atenúa la revisión del docente
Revisión del docente, justificantes, acumulados y reportes por alumno, grupo y periodo	Control de acceso físico a edificios o laboratorios

Incluye	No incluye
Análisis numérico del ausentismo (tendencia, proyección, cruce del límite y riesgo por alumno)	Piloto con grupos reales, medición del ahorro de tiempo y calibración con datos reales
Estación de aula: firmware, circuito y protocolo serie, diseñados y verificados en software	Pruebas de la estación con hardware físico (no se han hecho)
Programa en Node que maneja el puerto serie de la estación	Tabla de asociación entre el UID de la tarjeta y el alumno, y credenciales de las tarjetas
La estación solo lee la tarjeta y muestra el resultado	Registro de asistencia desde la estación en la base de datos (integración por funciones RPC pendiente)
Escenario de demostración con cámara y reloj simulados, solo en modo local	Cierre de sesiones desde la estación; no se hará, exige la autenticación del docente

Nota. El estado de cada elemento procede del repositorio del proyecto y de su documentación. La estación y su programa en Node son trabajo nuevo de este avance y no se han probado con hardware físico. Elaboración propia.

Supuestos.

- Los horarios, las aulas, las inscripciones y las cámaras se registran antes del periodo, porque una sesión solo se abre en el horario y el aula de su grupo.
- La computadora del docente está junto a la estación y conectada a ella por USB; en el modo local de demostración, la aplicación y la base de datos corren en esa misma computadora.
- Cada alumno que usa la estación tendría una tarjeta RFID personal; la entrega de tarjetas y su asociación con el alumno no están definidas todavía.
- El lector MFRC522 trabaja a 3.3 V, por lo que el circuito prevé divisores de resistencias para las señales de 5 V del Arduino UNO.
- El aviso de privacidad, el formato del consentimiento y su vigencia deben ser aprobados por el área jurídica de la universidad antes de un uso real; mientras tanto, quien no consiente se registra por la vía manual o, cuando exista la integración, por la tarjeta.
- Las cifras de tiempo y de límite de ausencias de la sección anterior son supuestos y estimaciones del análisis del proyecto, no mediciones institucionales.

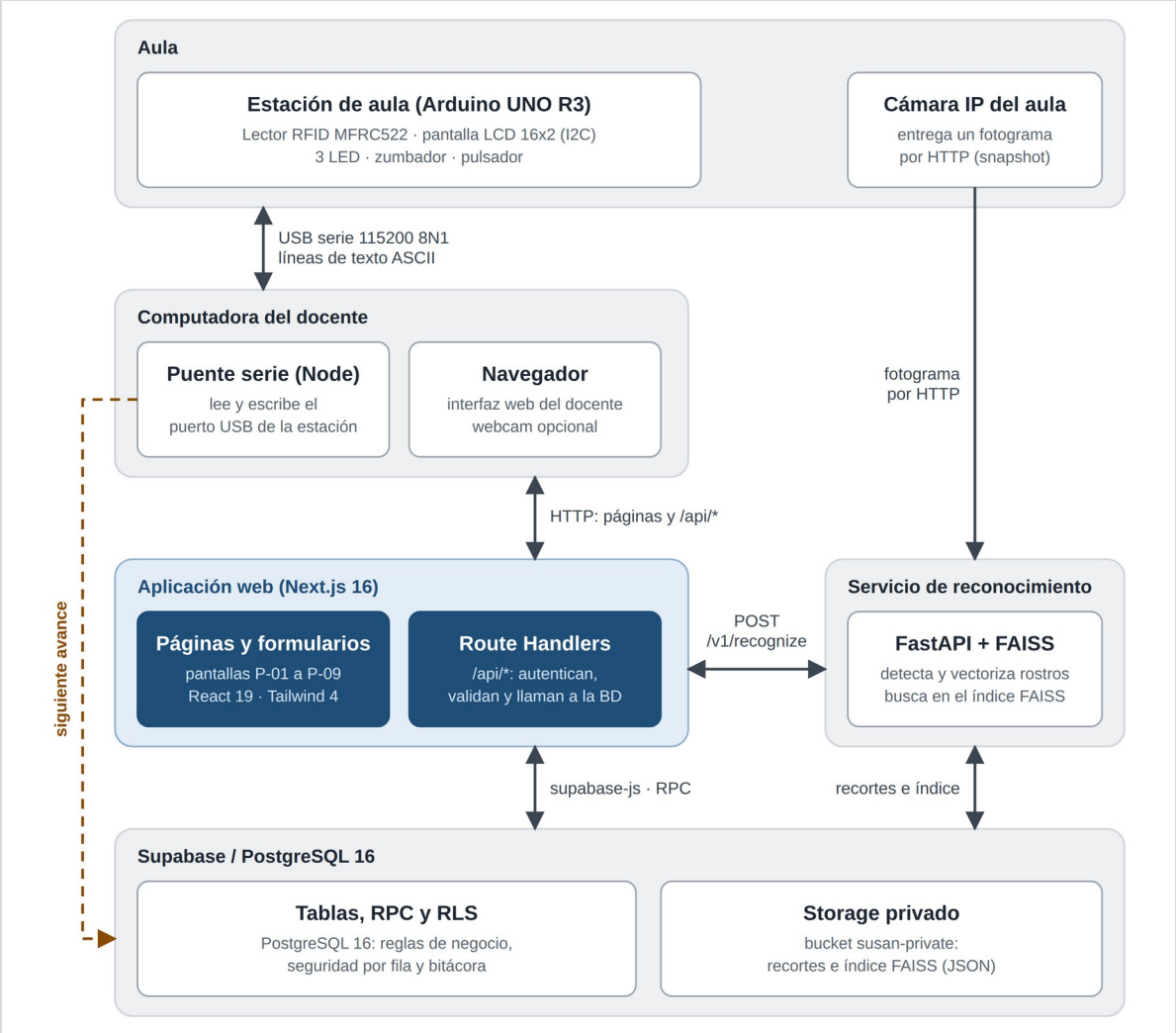
ARQUITECTURA GENERAL DEL SISTEMA.

El sistema se organiza en cuatro niveles unidos por enlaces de distinto tipo (Figura 1). En el aula están la estación de aula, con Arduino UNO R3, y la cámara IP. La estación se comunica por USB con el programa en Node de la computadora del docente mediante líneas de texto

ASCII (Arduino, s. f.-b). En esa misma computadora el docente usa el navegador para abrir la aplicación web, hecha con Next.js (Vercel, s. f.), cuyas páginas y Route Handlers son el núcleo del sistema: autentican al docente, validan cada entrada y llaman a las funciones de la base de datos con supabase-js. Para el reconocimiento facial, la aplicación llama por HTTP al servicio de reconocimiento (FastAPI y FAISS), que vectoriza los rostros del fotograma y los busca en un índice (Deng et al., 2019; Johnson et al., 2021). Los datos viven en PostgreSQL 16 de Supabase: tablas, funciones RPC, seguridad por filas (RLS) y un Storage privado (PostgreSQL Global Development Group, s. f.; Supabase, s. f.-a).

Figura 1.

Arquitectura general del sistema SUSAN.



Nota. Elaboración propia. La línea discontinua marca el enlace pendiente entre la estación y la base de datos; su ruta exacta (por un Route Handler o desde el navegador) está por decidir.

La estación solo conoce al puente serie y el navegador solo conoce a la aplicación web. La aplicación web es la que llama al servicio de reconocimiento y a las funciones de la base de datos, de modo que las reglas de negocio no viven en la interfaz. El enlace de la estación con el puente está diseñado y se probó solo en software; el de la estación con la base de datos, en línea discontinua, todavía no existe: hoy el puente reconoce la tarjeta con una tabla de demostración y no escribe nada en la base de datos.

1. Protocolo serie.

USB a 115200 baudios, 8 bits, sin paridad, 1 bit de paro (8N1). Todas las tramas son líneas de texto ASCII terminadas en salto de línea (`\n`; se tolera `\r\n`), de 64 bytes como máximo sin el salto, lo que permite probarlas con cualquier monitor serie. Al abrir el puerto, la placa se reinicia y envía `HELLO`.

Dirección	Trama	Significado	Ejemplo
Estación → PC	`HELLO estacion-aula 1.0`	Fin del arranque; se repite cada vez que la placa se reinicia	`HELLO estacion-aula 1.0`
Estación → PC	`UID <HEX>`	Tarjeta leída; HEX en mayúsculas y sin separadores (UID de 4, 7 o 10 bytes)	`UID 04A1B2C3`
Estación → PC	`BTN`	Pulsación del botón, que solo cambia la página de la pantalla	`BTN`
Estación → PC	`PONG`	Respuesta a `PING`	`PONG`
PC → estación	`PING`	Comprobación del enlace	`PING`
PC → estación	`SES <grupo>|<materia>|<presentes>/<total>`	Estado de la sesión para la pantalla; `SES ||0/0` indica que no hay sesión abierta	`SES 3o N|Programacion Ava|0/30`
PC → estación	`RES OK <nombre-corto>`	Tarjeta registrada: LED verde y un pitido corto	`RES OK Ana Demo`
PC → estación	`RES DUP [nombre-corto]`	Tarjeta ya registrada: LED ámbar y dos pitidos cortos	`RES DUP Ana Demo`
PC → estación	`RES NOREG`	Tarjeta no registrada: LED rojo y un pitido largo y grave	`RES NOREG`
PC → estación	`RES ERR [motivo]`	Error: LED rojo intermitente y sin sonido	`RES ERR`

Nota. Las tramas y el ejemplo de `UID` se tomaron de la documentación de la estación y se comprobaron contra las constantes y el analizador de comandos de `protocolo.h`. Los ejemplos de `SES` y `RES` son los que arma el puente con la sesión y las tarjetas de demostración, que son datos ficticios. Los textos se recortan a 16 columnas y se transliteran a ASCII porque la pantalla solo garantiza ASCII. Elaboración propia.

Una línea de más de 64 bytes, con caracteres de control, con un verbo desconocido o mal formada se descarta entera y se ignora; nunca se interpreta un fragmento. El mismo UID se ignora si la lectura anterior fue hace menos de 2 s, y además la tarjeta debe retirarse antes de que su siguiente lectura cuente. Si la computadora no envía ningún comando reconocido (`PING`, `SES` o `RES`) en 30 s, la estación muestra «Sin conexion» y enciende el LED ámbar de forma intermitente. Si después de un `UID` no llega `RES` en 3 s, la estación avisa por su cuenta con el LED rojo y el mensaje «Sin respuesta».

2. Secuencia de un registro por tarjeta.

La tarjeta RFID es la vía de registro del alumno que no otorgó su consentimiento para el reconocimiento facial, o de cualquier alumno cuando la cámara falla. La secuencia prevista es la siguiente; los pasos 4 y 5 son la integración con la base de datos y todavía no existen.

1. El alumno acerca su tarjeta al lector MFRC522; la estación solo lee el UID de la tarjeta.
2. El firmware descarta los rebotes, envía la trama `UID` y muestra «Tarjeta leida» en la pantalla mientras espera la respuesta.
3. El puente valida la línea y decide con lo que tiene en memoria: sin sesión abierta responde `RES ERR`, con un UID que no está en su tabla responde `RES NOREG` y con un alumno que ya figura presente responde `RES DUP`. Hoy la tabla UID-alumno es un archivo CSV con tarjetas ficticias y la sesión es un archivo JSON de demostración.
4. **A implementar.** El puente buscaría el UID en una tabla `credencial` (UID y alumno), que aún no existe, y registraría la asistencia con la función `api_registrar_manual`, la misma del pase de lista manual (RF-09). Hoy `registrarPresencia()` es un sustituto que solo escribe en pantalla.
5. **A implementar.** La función aplicaría las reglas del esquema: sesión abierta y alumno inscrito, que en una sesión automática sea sin consentimiento vigente o sin lecturas válidas, y un solo registro por alumno y sesión. Como la función exige al docente titular, la llamada debe hacerse con su sesión autenticada y sin guardar la clave `service_role` en el puente.
6. El puente responde `RES OK`, `RES DUP`, `RES NOREG` o `RES ERR` y la estación muestra el resultado con la pantalla, el LED y el zumbador; después el puente envía `SES` con los presentes y el total actualizados.

Esta secuencia valida la lógica, no el hardware. Se probó en la computadora de desarrollo: pruebas de la lógica y del sketch con hardware simulado (65 y 38 casos), el firmware

compilado para el UNO (14 122 bytes de memoria de programa y 1 227 de SRAM) y ejecutado en un simulador de instrucciones, y el puente (42 pruebas automáticas, 41 aprobadas y 1 omitida), con un recorrido completo por un puerto serie virtual. Falta probar con la placa, el lector y tarjetas reales, en particular la regla de retirada de la tarjeta, que depende de cómo respondan las tarjetas físicas.

3. Secuencia del pase de lista con cámara.

Este es el flujo principal de SUSAN. Está implementado en la aplicación y cubierto por pruebas automáticas; su demostración usa una cámara y un reloj simulados, solo en modo local, y no se ha medido con grupos reales.

1. El docente abre la sesión de clase en la aplicación web. La función `api_abrir_sesion` solo lo permite dentro del horario del grupo y fija el modo de registro: automático si el aula tiene al menos una cámara activa, manual en caso contrario (RN-06).
2. En modo automático y durante la ventana de tolerancia, la página de la sesión envía `POST /api/sesiones/{id}/captura` cada `intervalo_muestreo_seg`: con el fotograma de la cámara del navegador o, para una cámara IP, sin imagen, porque el servicio de reconocimiento toma el fotograma de la cámara.
3. El Route Handler autentica al docente y `api_sesion_monitor` confirma que es el titular del grupo, que la sesión está abierta, es automática y sigue en ventana, y que la cámara es del aula y está activa.
4. `svc_alumnos_candidatos` entrega los identificadores FAISS de los alumnos inscritos con consentimiento vigente: quien no está inscrito o no consintió nunca es candidato (RF-04, RNF-06).
5. El servicio de reconocimiento (`POST /v1/recognize`) detecta los rostros, calcula el vector de cada uno solo en memoria, busca en el índice FAISS restringido a esos candidatos, sube los recortes al Storage privado y devuelve para cada rostro el candidato y su similitud, sin el vector. Si el servicio falla, la aplicación responde 503 sin escribir en la base de datos y la sesión sigue operable en manual.
6. `svc_crear_captura` y `svc_registrar_detecciones` guardan la captura y las detecciones: aplican el umbral de similitud y las reglas RN-02 a RN-05, y crean o actualizan un solo registro de asistencia por alumno y sesión.
7. El docente sigue el avance en la pantalla de la sesión (P-03) y puede registrar a mano a quien no consintió o no fue reconocido.
8. Al cerrar la sesión se consolidan las lecturas, se confirma presente o retardo a quien alcanzó el mínimo de lecturas válidas, se crea el registro de ausente de cada inscrito activo sin registro, se anota la bitácora y se recalculan los acumulados. Los alumnos con pocas lecturas quedan pendientes en la cola de revisión (P-05), donde el docente los

confirma o corrige. El cierre lo hace el docente con `api_cerrar_sesion` o la tarea `/api/jobs/cerrar-sesiones`.

Nota. La captura la dispara el navegador del docente, no el servidor: si cierra la pestaña, el muestreo se detiene. La tarea de cierre automático existe, pero ningún programador de tareas del repositorio la ejecuta todavía. La ruta de la cámara IP solo se probó con pruebas automáticas, nunca con una cámara real. Elaboración propia.

DISEÑO DEL CIRCUITO DE LA ESTACIÓN DE AULA.

La estación de aula es un dispositivo de escritorio que se conecta por USB a la computadora del docente y permite pasar lista con una tarjeta RFID personal. Existe para el alumno que no otorgó su consentimiento para el reconocimiento facial, que lo revocó, o para cualquier alumno cuando falla la cámara del aula: en SUSAN no se reconoce ningún rostro sin consentimiento vigente (RN-13, RNF-06), y la tarjeta ofrece una vía de registro que no usa el rostro. Es la parte de hardware digital del proyecto integrador y se desarrolló en la unidad «Manejo de puertos» de Programación Avanzada. No es una autenticación fuerte, porque una tarjeta se puede prestar o clonar, y no cierra sesiones: cerrarlas exige la autenticación del docente en la aplicación web.

Este capítulo describe el diseño eléctrico. El circuito se diseñó y se dibujó, pero no se ha armado ni probado con componentes físicos; los consumos y los niveles de voltaje son cálculos con valores típicos de hojas de datos, no mediciones.

1. Tarjeta propuesta: Arduino UNO R3.

Se propone la tarjeta Arduino UNO R3 (microcontrolador ATmega328P, lógica de 5 V, reloj de 16 MHz). Las razones que respaldan la elección son las siguientes:

- Tiene puerto USB-B con conversor USB-serie, por lo que la misma conexión alimenta la placa y la comunica con la computadora del docente a 115200 baudios, 8N1.
- Expone el bus SPI (D10 a D13) que usa el lector MFRC522 y el bus I2C (A4 y A5) que usa la pantalla LCD.
- Su regulador entrega 3.3 V hasta 150 mA, suficientes para el módulo lector, que consume 30 mA como máximo.
- Dispone de los pines digitales que se necesitan para tres LED, un zumbador y un pulsador, y deja libres D3, D4 y A0 a A3.
- Su memoria alcanza para el programa: el firmware compilado ocupa 14 122 B de 32 256 B de memoria de programa (43.1 %) y 1 227 B de 2 048 B de SRAM (59.9 %).

2. Bloques del circuito.

El circuito se divide en bloques funcionales; las referencias (U1, R1, etc.) son las del esquema y de la lista de materiales.

Bloque	Componentes	Función
Control y enlace	U1 Arduino UNO R3	Ejecuta el firmware; se alimenta y se comunica por USB
Lector RFID	U2 MFRC522, C1, C2	Lee el UID de la tarjeta (13.56 MHz) por SPI
Adaptación de niveles	R1 a R4 (1 kΩ), R5 a R8 (2 kΩ)	Baja SS, SCK, MOSI y RST de 5 V a 3.3 V
Pantalla	U3 LCD 16×2 con mochila I2C, C3	Muestra sesión, resultado y diagnóstico
Indicadores luminosos	LED1 a LED3, R9 a R11 (220 Ω)	Verde: OK; ámbar: duplicado o sin conexión; rojo: no registrada o error
Indicador sonoro	BZ1, R12 (100 Ω)	Pitidos de resultado y de arranque
Entrada manual	SW1	Cambia la página de la pantalla; nunca cierra sesiones

Nota. Elaboración propia, a partir de la lista de materiales y de la tabla de pines de la estación.

3. Asignación de pines.

La tabla siguiente reproduce las conexiones de `pines.csv`. Se omiten las columnas del puerto AVR, el sentido de la señal y las notas, que están en el archivo; el pin IRQ del módulo no se conecta y no aparece en la tabla.

Id	Pin UNO	Componente	Pin del componente	Nivel	Cable
`SPI-SS`	D10	U2 MFRC522	SDA (SS)	5 V → 3.3 V	violeta
`SPI-SCK`	D13	U2 MFRC522	SCK	5 V → 3.3 V	dorado
`SPI-MOSI`	D11	U2 MFRC522	MOSI	5 V → 3.3 V	azul
`SPI-MISO`	D12	U2 MFRC522	MISO	3.3 V directo	verde
`RC522-RST`	D9	U2 MFRC522	RST	5 V → 3.3 V	gris
`RC522-3V3`	3V3	U2 MFRC522	3.3V	3.3 V	naranja
`RC522-GND`	GND	U2 MFRC522	GND	0 V	negro
`LCD-SDA`	A4	U3 LCD I2C	SDA	5 V	cian
`LCD-SCL`	A5	U3 LCD I2C	SCL	5 V	rosa

Id	Pin UNO	Componente	Pin del componente	Nivel	Cable
`LCD-5V`	5V	U3 LCD I2C	VCC	5 V	rojo
`LCD-GND`	GND	U3 LCD I2C	GND	0 V	negro
`LED-VERDE`	D5	R9 220 Ω + LED1	ánodo (vía R9)	5 V	verde azulado
`LED-AMBAR`	D6	R10 220 Ω + LED2	ánodo (vía R10)	5 V	verde azulado
`LED-ROJO`	D7	R11 220 Ω + LED3	ánodo (vía R11)	5 V	verde azulado
`ZUMBADOR`	D8	R12 100 Ω + BZ1	terminal + (vía R12)	5 V	verde azulado
`BOTON`	D2	SW1	terminal 1	5 V (pull-up) / 0 V	café
`GND-DER`	GND	LED1-3 cátodos + BZ1 − + SW1 terminal 2	GND	0 V	negro

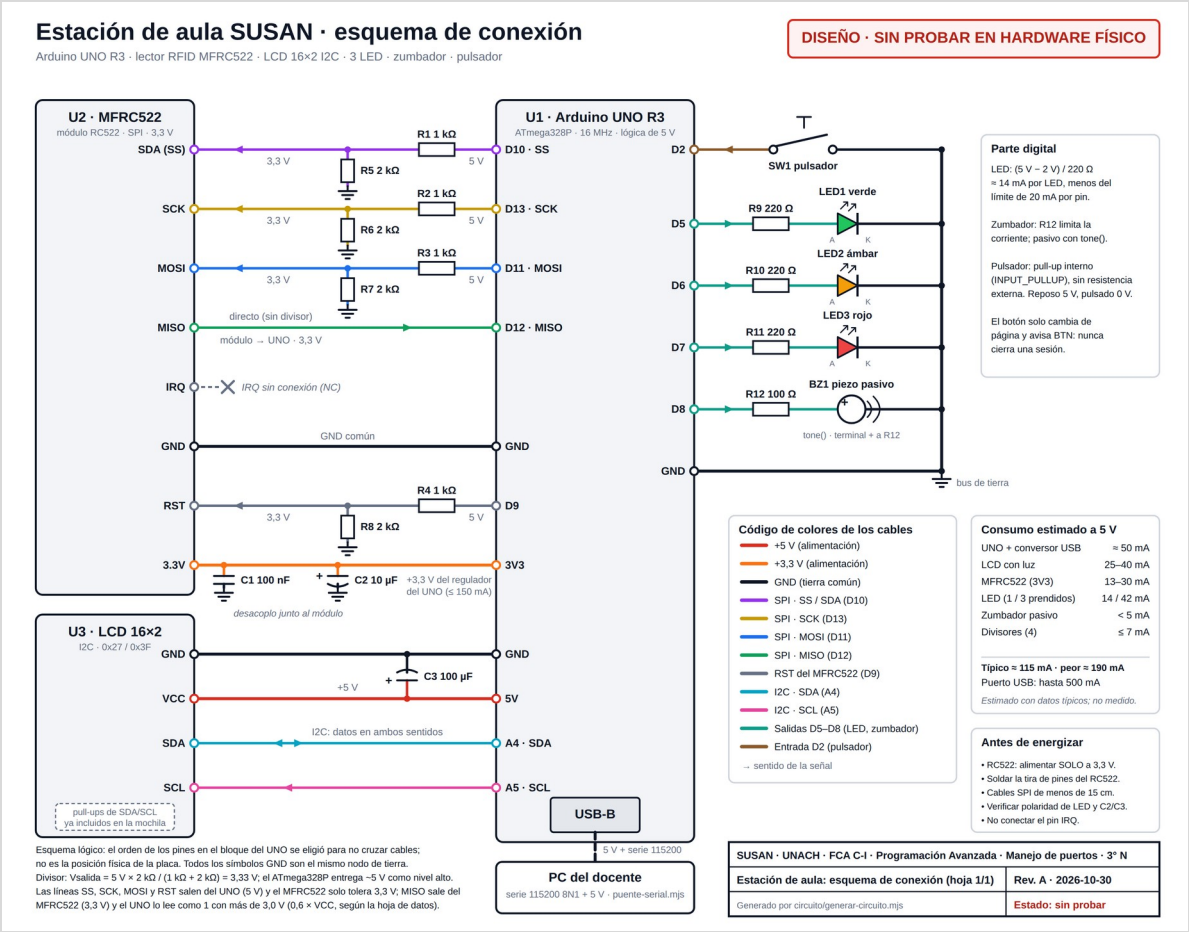
Nota. Valores tomados de `circuito/pines.csv`; 17 de las 18 filas del archivo (se omite `RC522-IRQ`, sin conexión). Elaboración propia.

4. Esquema del circuito.

El esquema se genera con el programa `generar-circuito.mjs`, que falla si el dibujo deja de coincidir con `pines.csv` y `bom.csv`; además se revisó a simple vista.

Figura 2.

Esquema de conexión de la estación de aula.



Nota. Elaboración propia. Diseño no probado en hardware físico.

El módulo MFRC522 funciona a 3.3 V y sus entradas no toleran 5 V, mientras que el UNO entrega señales de 5 V. Por eso SS, SCK, MOSI y RST (UNO hacia módulo) pasan por divisores resistivos de 1 kΩ en la parte alta y 2 kΩ en la parte baja, con lo que la salida es 5 × 2/(1 + 2) = 3.33 V. El README de la estación indica medir con un multímetro que cada nodo marque aproximadamente 3.3 V antes de conectar el módulo. La línea MISO (módulo hacia UNO) va directa: el UNO lee 3.3 V como nivel alto porque su umbral es 0.6 × VCC = 3.0 V, de modo que el margen es de apenas 0.3 V; si las lecturas fueran inestables, el propio README recomienda un traductor de niveles bidireccional en esa línea.

El consumo es una estimación, no una medición. Por componente se estiman: UNO con conversor USB, 50 mA; LCD con luz, de 25 a 40 mA; MFRC522, de 13 a 30 mA desde el regulador de 3.3 V; cada LED, 14 mA (42 mA los tres); zumbador pasivo, menos de 5 mA; y los cuatro divisores, hasta 7 mA. El total es de unos 115 mA típicos y 190 mA en el peor caso,

por debajo de los 500 mA de un puerto USB 2.0. El regulador de 3.3 V del UNO entrega hasta 150 mA y el módulo usa 30 mA o menos. Para estabilizar la alimentación se colocan C1 (100 nF) y C2 (10 µF) junto al módulo y C3 (100 µF) junto a la pantalla.

5. Lista de materiales.

Refs.	Cant.	Descripción	Valor	En esquema
U1	1	Arduino UNO R3 (o compatible ATmega328P) con cable USB A-B	5 V / 16 MHz	sí
U2	1	Módulo lector RFID MFRC522 (RC522) 13.56 MHz	3.3 V	sí
U3	1	Pantalla LCD 16×2 HD44780 con mochila I2C PCF8574	5 V	sí
R1-R4	4	Resistor 1/4 W (parte alta de los divisores)	1 kΩ	sí
R5-R8	4	Resistor 1/4 W (parte baja de los divisores)	2 kΩ	sí
R9-R11	3	Resistor 1/4 W en serie con cada LED	220 Ω	sí
R12	1	Resistor 1/4 W en serie con el zumbador	100 Ω	sí
LED1	1	LED 5 mm verde	verde	sí
LED2	1	LED 5 mm ámbar	ámbar	sí
LED3	1	LED 5 mm rojo	rojo	sí
BZ1	1	Zumbador piezoeléctrico pasivo (o activo de 5 V ≤ 20 mA)	5 V	sí
SW1	1	Pulsador táctil 6×6 mm normalmente abierto	-	sí
C1	1	Capacitor cerámico	100 nF	sí
C2	1	Capacitor electrolítico 16 V	10 µF	sí
C3	1	Capacitor electrolítico 16 V	100 µF	sí
TARJETAS	3	Tarjetas o llaveros MIFARE Classic 1K (13.56 MHz)	-	no
PROTOBOARD	1	Protoboard de 830 puntos	-	no
JUMPERS	30	Cables jumper (aprox. 20 macho-macho + 10 macho-hembra)	-	no

Refs.	Cant.	Descripción	Valor	En esquema
CAJA	1	Caja o soporte impreso (opcional)	-	no

Nota. Valores tomados de `circuito/bom.csv` (19 filas). Las filas que no están en el esquema son tarjetas, protoboard, cables y caja; la caja es opcional y está pendiente porque no se diseñó. Elaboración propia.

CÓDIGO DE ARDUINO.

El programa de la estación se divide en tres partes: el *sketch* de Arduino (`estacion_aula.ino`), que maneja los pines, el lector y la pantalla; el archivo de cabecera `protocolo.h`, que contiene la lógica pura (análisis de tramas, antirrebotes, temporizaciones, efectos y texto de la pantalla) sin depender de ninguna librería de Arduino, y el programa del puerto serie en la computadora (`puente-serial.mjs`, en Node), que atiende las tarjetas leídas. Separar la lógica pura permite probarla en la computadora de desarrollo con un compilador común. Los fragmentos se citan sin modificaciones.

1. Archivos del diseño.

Archivo	Contenido	Líneas
`firmware/estacion_aula/estacion_aula.ino`	Programa principal: pines, lector, pantalla, serie y ciclo `loop()`	381
`firmware/estacion_aula/protocolo.h`	Lógica pura: tramas, antirrebotes, efectos y pantallas	823
`puente/puente-serial.mjs`	Programa del puerto serie en el PC (Node)	710
`firmware/pruebas/test_protocolo.cpp`	Pruebas de la lógica de `protocolo.h`	839
`firmware/pruebas/test_sketch.cpp`	Escenarios del sketch con hardware simulado	643
`firmware/pruebas/simavr/simavr_estacion.c`	Ejecución del binario en el simulador simavr	265
`puente/pruebas/puente.test.mjs`	Pruebas automáticas del puente	514
`puente/pruebas/puente-pty.test.mjs`	Puente contra un puerto serie virtual	103
`firmware/compilar-avr.sh`	Compilación manual para el UNO	121

Nota. Conteo con `wc -l`. Elaboración propia.

2. Protocolo serie.

La estación y el PC se comunican por USB a 115200 baudios, 8N1, con líneas de texto ASCII terminadas en `\n` (se tolera `\r\n`) de 64 bytes como máximo sin el salto. Al abrir el puerto, la placa se reinicia y envía `HELLO`.

Dirección	Trama	Significado	Ejemplo
Estación → PC	`HELLO estacion-aula 1.0`	Fin del arranque; se repite cada vez que la placa se reinicia	`HELLO estacion-aula 1.0`
Estación → PC	`UID <HEX>`	Tarjeta leída; HEX en mayúsculas y sin separadores (UID de 4, 7 o 10 bytes)	`UID 04A1B2C3`
Estación → PC	`BTN`	Pulsación del botón, que solo cambia la página de la pantalla	`BTN`
Estación → PC	`PONG`	Respuesta a `PING`	`PONG`
PC → estación	`PING`	Comprobación del enlace	`PING`
PC → estación	`SES <grupo>|<materia>|<presentes>/<total>`	Estado de la sesión para la pantalla; `SES ||0/0` indica que no hay sesión abierta	`SES 3o N|Programacion Ava|0/30`
PC → estación	`RES OK <nombre-corto>`	Tarjeta registrada: LED verde y un pitido corto	`RES OK Ana Demo`
PC → estación	`RES DUP [nombre-corto]`	Tarjeta ya registrada: LED ámbar y dos pitidos cortos	`RES DUP Ana Demo`
PC → estación	`RES NOREG`	Tarjeta no registrada: LED rojo y un pitido largo y grave	`RES NOREG`
PC → estación	`RES ERR [motivo]`	Error: LED rojo intermitente y sin sonido	`RES ERR`

Nota. Las tramas y el ejemplo de `UID` se tomaron de la documentación de la estación y se comprobaron contra las constantes y el analizador de comandos de `protocolo.h`. Los ejemplos de `SES` y `RES` son los que arma el puente con la sesión y las tarjetas de demostración, que son datos ficticios. Elaboración propia.

3. Lógica del protocolo.

El analizador de `protocolo.h` acepta solo las formas exactas `PING`, `SES ...` y `RES ...`. Cualquier otra línea (más de 64 bytes, verbo desconocido, mal formada o en minúsculas) se ignora sin consecuencias y no cuenta como señal de vida de la computadora. La función `despuesDePalabra` exige la palabra completa, de modo que `RES OKAY` no se interpreta como `RES OK`.

Código C++.

```
inline bool analizarRes(const char* r, Comando& c) {
    static const char* const PALABRAS[] = {"OK", "DUP", "NOREG", "ERR"};
    static const Tipo TIPOS[] = {CMD_RES_OK, CMD_RES_DUP, CMD_RES_NOREG,
CMD_RES_ERR};
    for (uint8_t i = 0; i < 4; i++) {
        const char* resto = despuesDePalabra(r, PALABRAS[i]);
        if (resto) {
            ajustar(resto, c.texto1, COLS);
            c.tipo = TIPOS[i];
            return true;
        }
    }
    return false;
}

// Interpreta una línea ya ensamblada. Devuelve true si es un comando conocido y
// bien formado; en cualquier otro caso devuelve false y la línea se ignora.
inline bool analizar(const char* linea, Comando& c) {
    c.tipo = CMD_NINGUNO;
    c.texto1[0] = '\0';
    c.texto2[0] = '\0';
    c.presentes = 0;
    c.total = 0;
    if (!linea) {
        return false;
    }
    size_t n = strlen(linea);
    if (n == 0 || n > MAX_LINEA) {
        return false;
    }
    char l[MAX_LINEA + 1];
    memcpy(l, linea, n + 1);
    while (n > 0 && l[n - 1] == ' ') {
        l[--n] = '\0';
    }
```

```
    if (n == 0) {
        return false;
    }
    if (strcmp(l, "PING") == 0) {
        c.tipo = CMD_PING;
        return true;
    }
    if (empiezaCon(l, "SES ")) {
        return analizarSes(l + 4, c);
    }
    if (empiezaCon(l, "RES ")) {
        return analizarRes(l + 4, c);
    }
    return false;
}
```

Nota. Fragmento de `protocolo.h` (líneas 333 a 381): interpretación de las respuestas `RES` y análisis de una línea ya ensamblada. Elaboración propia.

4. Programa principal.

El programa principal es no bloqueante: `loop()` no usa `delay()` y compara el tiempo con `millis()` en aritmética modular. En cada vuelta se lee el puerto serie, el botón y el lector, se vigila el plazo de la respuesta de la computadora, se avanzan los efectos (LED y zumbador) y se redibuja la pantalla. La función `setup()` termina enviando `HELLO`, con lo que la computadora sabe que ya puede mandar la sesión.

Código C++.

```
void setup() {
    pinMode(PIN_BOTON, INPUT_PULLUP);
    pinMode(PIN_LED_VERDE, OUTPUT);
    pinMode(PIN_LED_AMBAR, OUTPUT);
    pinMode(PIN_LED_ROJO, OUTPUT);
    pinMode(PIN_ZUMBADOR, OUTPUT);
    escribirLeds(0);

    Serial.begin(proto::BAUDIOS);
    SPI.begin();
    Wire.begin();
    Wire.setWireTimeout(25000UL, true);  // un bus I2C bloqueado no cuelga la
estación

    proto::modeloInicial(modelo);
    mostrado[0][0] = '\0';
```

```
    mostrado[1][0] = '\0';
    efectoTexto[0] = '\0';

    iniciarPantalla();
    iniciarLector();

    const proto::ms_t ahora = millis();
    arranque = ahora;
    ultimoRx = ahora;
    tSondeo = ahora;
    tRedibujo = ahora;
    tMantenimiento = ahora;
    tRefresco = ahora;
    boton.reiniciar(digitalRead(PIN_BOTON) == LOW, ahora);
    iniciarEvento(proto::EV_ARRANQUE, "", ahora);

    // Al final de setup(): al abrir el puerto el UNO se reinicia (DTR) y este
    // HELLO le avisa a la PC que ya puede enviar la sesión.
    enviarLinea(proto::HELLO);
}

void loop() {
    const proto::ms_t ahora = millis();
    enlace = proto::estadoEnlace(huboRx, ahora, ultimoRx, arranque);
    leerSerie(ahora);
    atenderBoton(ahora);
    atenderLector(ahora);
    if (esperandoRes && proto::respuestaVencida(ahora, tEnvio)) {
        esperandoRes = false;  // la PC no contestó: se avisa localmente
        iniciarEvento(proto::EV_ERR, "Sin respuesta", ahora);
    }
    avanzarEfectos(ahora);
    mantenimiento(ahora);
    dibujar(ahora);
}
```

Nota. Fragmento de `estacion_aula.ino` (líneas 332 a 381): `setup()` y `loop()`. Elaboración propia.

5. Programa del puerto serie en el PC.

El puente en Node abre el puerto serie, convierte las líneas `UID` en decisiones y responde con `RES`. Aplica el orden rebote, sesión, tarjeta registrada, duplicado y registro. La función de registro, `registrarPresencia()`, es hoy un sustituto que solo escribe en pantalla; no escribe en la base de datos.

Código JavaScript.

```
    // Decisión por tarjeta. El orden importa: rebote -> sesión -> registrada ->
    // duplicado -> registro.
    async manejarUid(uid) {
        const ahora = this.reloj();
        if (this.ultimoUid === uid && ahora - this.ultimoUidT < ANTIREBOTE_UID_MS) {
            this.log('depurar', `UID ${this.uid(uid)} repetido en menos de $
{ANTIREBOTE_UID_MS / 1000} s: ignorado`);
            return;
        }
        this.ultimoUid = uid;
        this.ultimoUidT = ahora;

        if (!this.sesion.abierta) {
            this.log('info', `UID ${this.uid(uid)}: no hay sesión abierta`);
            this.enviar(armarRes('ERR', 'Sin sesion abierta'));
            return;
        }
        const cred = this.credenciales.get(uid);
        if (!cred) {
            this.log('info', `UID ${this.uid(uid)}: tarjeta no registrada`);
            this.enviar(armarRes('NOREG'));
            return;
        }
        const alumno = this.verboso ? `${cred.nombreCorto} (${cred.matricula})` :
`alumno ${enmascararMatricula(cred.matricula)}`;
        if (this.sesion.presentes.has(cred.matricula) ||
this.enCurso.has(cred.matricula)) {
            this.log('info', `UID ${this.uid(uid)} (${alumno}): ya estaba
presente`);
            this.enviar(armarRes('DUP', cred.nombreCorto));
            return;
        }
        this.enCurso.add(cred.matricula);
        try {
            const r = await this.registrar(cred.matricula, { log: this.log, verboso:
this.verboso, uid });
            if (!r || r.ok !== true) throw new Error('el registro de presencia no se
confirmó');
            this.sesion.presentes.add(cred.matricula);
        } catch (e) {
            this.log('error', `UID ${this.uid(uid)} (${alumno}): no se pudo
registrar: ${e.message}`);
            this.enviar(armarRes('ERR'));
```

```
            return;
        } finally {
            this.enCurso.delete(cred.matricula);
        }
        this.log('info', `UID ${this.uid(uid)} (${alumno}): presente ($
{this.sesion.presentes.size}/${this.sesion.total})`);
        this.enviar(armarRes('OK', cred.nombreCorto));
        this.sincronizar(); // actualiza presentes/total en la LCD
    }
```

Nota. Fragmento de `puente-serial.mjs` (líneas 377 a 420): método `manejarUid`. Elaboración propia.

6. Estado de verificación.

Todas las verificaciones se hicieron en la computadora de desarrollo, con herramientas de software. Los resultados que registran los archivos de la estación son estos:

Verificación	Herramienta	Resultado
Compilación para UNO	avr-gcc 7.3.0, núcleo Arduino AVR 1.8.6, librerías MFRC522 1.4.12 y LiquidCrystal_I2C 1.1.4	Sin errores ni avisos propios (-Wall -Wextra); flash 14 122 B (43.1 %), SRAM 1 227 B (59.9 %)
Lógica de `protocolo.h`	Pruebas en el PC, también con sanitizadores	65 casos, 0 fallidos (6 138 afirmaciones)
Sketch con hardware simulado	Pruebas en el PC, también con sanitizadores	38 casos, 0 fallidos
Mutaciones del firmware	Cambios intencionales	Las 11 mutaciones fueron detectadas
Binario en simulador de instrucciones	simavr 1.6, ATmega328P a 16 MHz	12 comprobaciones, 0 fallidas
Puente en Node	Pruebas automáticas y puerto serie virtual	42 pruebas: 41 aprobadas, 1 omitida, 0 fallidas

Nota. Datos de `firmware/COMPILACION.txt`, `firmware/pruebas/RESULTADO.txt` y `puente/RESULTADO.txt` (registros del 30 de octubre de 2026). Elaboración propia.

El firmware se compiló con un procedimiento manual que replica las banderas del núcleo 1.8.6, porque no fue posible obtener `arduino-cli` en el entorno de desarrollo, y el tamaño

exacto puede variar unos bytes con el compilador del IDE. El simulador de instrucciones es un modelo mínimo, sin pantalla ni tarjetas RFID y con el lector reducido a un esclavo SPI que solo contesta el registro de versión. El puente se probó con un dispositivo en memoria y con pseudoterminales, sin ninguna placa. Estos resultados validan la lógica del programa en software y simulación; **no validan el hardware físico**. Falta probar el cableado y los divisores de voltaje, las lecturas reales de tarjetas (incluida la detección de retirada), la dirección I2C de la pantalla (0x27 o 0x3F), el antirrebote real del pulsador y el puerto serie real. La integración con la base de datos (tabla de credenciales y función RPC) es el siguiente avance.

DISEÑO Y CÓDIGO DE LOS FORMULARIOS.

En SUSAN los formularios son formularios web y flujos guiados de la aplicación, construidos con las mismas primitivas de campo y de contenedor. El inventario hecho sobre `web/src` (commit `f742bb4` de `origin/main`) registra **41 formularios y flujos**: 4 de acceso, 7 de alumnos, 7 de catálogos y cuentas, 9 de grupos, 1 del portal del alumno, 5 del docente, 3 de filtros, 3 de la consola técnica y 2 que existen solo en el modo local de desarrollo. Además hay 4 controles de entrada que no son formularios (por ejemplo, la caja de búsqueda de la paleta de comandos).

Según su contenedor, **2 son asistentes** construidos sobre `ui/Asistente` (alta de alumno y alta de grupo), **5 son flujos guiados con guion propio** (activar cuenta, importar CSV, «Mi registro facial», registro facial guiado de cinco tomas y registrar justificante), **5 son altas de un paso en un panel lateral** (aula, materia, docente, cuenta de usuario y cámara) y el resto son formularios de página, diálogos con campos y barras de filtros. Esta sección describe cuáles son, qué principios de diseño los rigen, cómo se ven, cómo está organizado su código y qué parte de ellos se verificó.

1. Formularios del sistema.

La Tabla 1 resume doce de los formularios más representativos. El inventario completo, con cada campo, sus mensajes de validación, la función a la que llama y sus pruebas, se entrega en los anexos.

Formulario	Campos principales	Ruta
Inicio de sesión	Correo institucional, contraseña	`/login`
Recuperar contraseña	Correo institucional	`/recuperar`
Activar cuenta (4 pasos)	Contraseña, color de avatar, aviso de privacidad	`/activar-cuenta`

Formulario	Campos principales	Ruta
Alta de alumno (3 pasos)	Matrícula, nombres, apellidos, correo, cuenta	`/admin/alumnos/nuevo`
Datos del alumno	Los de alta y el estatus	`/admin/alumnos/[id]`
Aula (panel lateral)	Clave, edificio, capacidad	`/admin/aulas`
Materia (panel lateral)	Clave, nombre, créditos, horas por semana	`/admin/materias`
Docente (panel lateral)	Número de empleado, correo, apellidos, nombres	`/admin/docentes`
Nueva cuenta de usuario	Rol, docente o alumno vinculado, correo de acceso	`/admin/usuarios`
Alta de grupo (3 pasos)	Materia, docente, aula, clave, periodo, horario	`/admin/grupos/nuevo`
Importar inscripciones (4 pasos)	Archivo CSV	`/admin/grupos/[id]`, pestaña Inscripciones
Registrar justificante	Falta, motivo, documento	`/alumno`

Nota. Tabla 1. Elaboración propia con base en el inventario de formularios de `web/src` (commit `f742bb4`).

2. Principios de diseño.

El diseño de los formularios se destiló de los documentos de `docs/design/formularios/` (`BRIEF-COMUN-FRENTES.md`, `PRIMITIVAS-campos.md`, `PRIMITIVAS-contenedores.md`, `auditoria.md`, `referencias-campos-y-validacion.md`, `referencias-flujos-guiados.md` y los frentes `FRENTE-F2` a `FRENTE-F6`) y de `docs/design/SISTEMA.md`. La auditoría del 6 de octubre de 2026 encontró que los formularios de entonces validaban solo al enviar, no avisaban de cambios sin guardar, usaban selectores nativos de hasta 200 opciones, no declaraban los límites de la base en los campos y duplicaban errores en alerta y aviso emergente. Con ese diagnóstico se fijó la dirección de `SISTEMA.md`: todo formulario es guiado y dinámico, y el rediseño es solo capa de presentación y validación de conveniencia; no cambia las funciones RPC, las rutas, los payloads ni los códigos de error, de modo que el servidor sigue siendo la validación real. Los principios resultantes son los siguientes.

- **Un campo, una anatomía.** `ui/Field` envuelve un único control y lo conecta con su texto mediante `id`, `aria-describedby` y `aria-invalid`. Presenta etiqueta, control, ayuda con

ejemplo, error con icono y `role="alert"`, estado «Válido» y contador `n/max`. Lo obligatorio se marca con `*` y lo opcional con la palabra «(opcional)»; nunca se mezclan ambas marcas.

- **El error dice qué se espera y cómo arreglarlo.** No se usan mensajes genéricos como «valor inválido»; por ejemplo: «Escribe los créditos: un número entero de 1 a 99 (no puede ser 0).».
- **Se valida al salir del campo o al enviar**, nunca la primera vez que se escribe; con un error visible se revalida en cada tecla. Enviar con datos inválidos no llama al servidor y enfoca el primer campo inválido.
- **El botón de alta no se apaga por datos inválidos**: el clic muestra los errores. Solo se apaga en edición sin cambios, y entonces dice el motivo junto al botón.
- **Un solo aviso por error y pegado a su campo.** Los códigos del servidor (`DUPLICADO`, `HORARIO_CHOQUE`, entre otros) se convierten en un mensaje bajo el campo que los causó; tras un error se conserva todo lo capturado.
- **Los límites son los de la base.** `lib/formularios/limites.js` declara `LIMITES` y `PATRONES` (por ejemplo, matrícula de 10 caracteres) y una prueba los compara con el DDL de la base.
- **Contenedor según la tarea.** Un paso de hasta unos seis campos va en `PanelLateral`; los datos con dependencias o que conviene revisar antes de crear van en `Asistente` con una pantalla de revisión; un asistente para dos o tres campos se considera un anti-patrón, por lo que el justificante son tres bloques numerados.
- **Nunca volver en silencio.** Al terminar, `ui/SuccessState` muestra el resultado y de dos a cuatro siguientes pasos reales.
- **Cambios sin guardar.** `FormActions` muestra «N cambios sin guardar» con punto de color y texto, y `useCambiosSinGuardar` avisa al recargar o cerrar la página.
- **Accesibilidad y móvil.** Nada se expresa solo con color (error = borde, icono y texto); el foco es visible; los diálogos atrapan y devuelven el foco; toda pantalla debe verse bien a 360 px, sin desborde horizontal y respetando «reducir movimiento» (regla de `CLAUDE.md`).
- **Forma y tokens.** Campos y botones de 8 px de radio, tarjetas sin sombra, botón primario negro, interfaz en Hanken Grotesk de 14 px y colores solo mediante tokens semánticos; los iconos son siempre de Lucide.

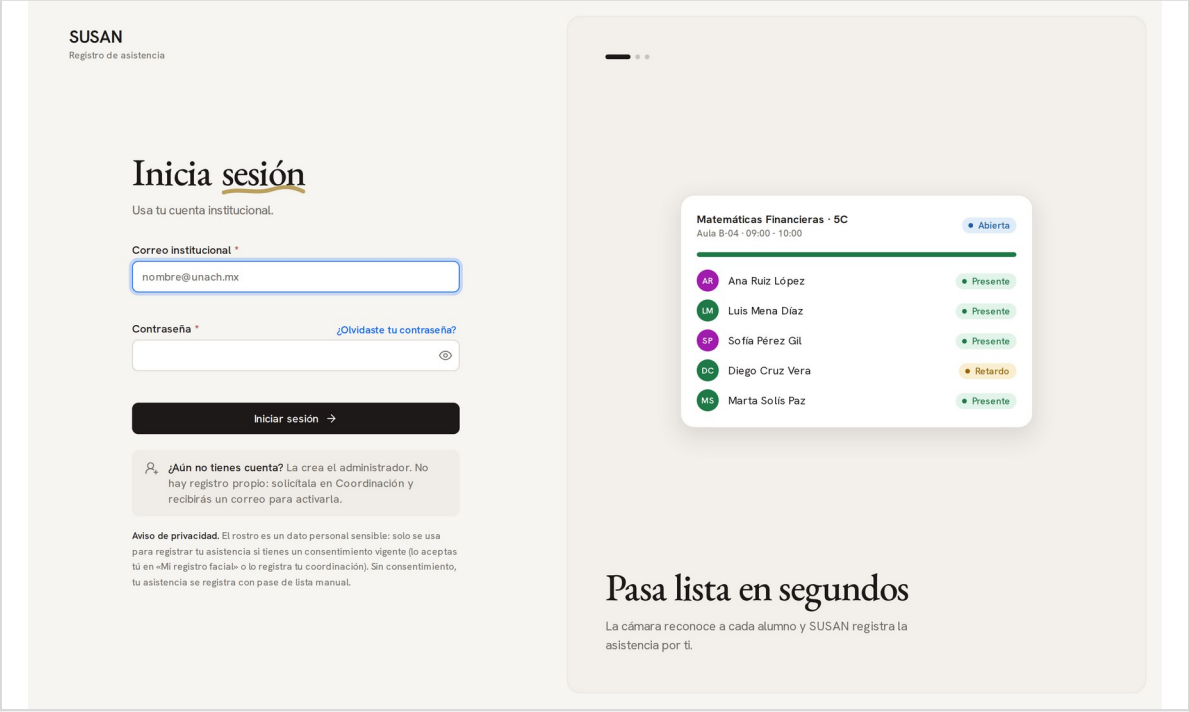
Las referencias de interfaz provienen de capturas de Mobbin (R01 a R18, con su dirección en `referencias-campos-y-validacion.md`), de las que se extrajeron 12 patrones y 14 anti-patrones a evitar; en este reporte no se reproducen imágenes de terceros.

3. Diseño de las pantallas.

Las capturas se tomaron de la aplicación en ejecución (Next 16.3.8 en modo de desarrollo, `SUSAN_LOCAL=1`, reloj de demostración fijo en el lunes 14 de septiembre de 2026, 07:12) con Playwright y Chromium, en escritorio de 1440x900 a factor 2 y en móvil de 390x844 a factor 3, con «reducir movimiento» activado. Los registros creados por las capturas llevan el prefijo «ZZ».

Figura 3.

Pantalla de inicio de sesión en estado vacío.



Nota. Ruta `/login`; el foco inicial está en el correo y el ejemplo `nombre@unach.mx` aparece como texto de apoyo. Elaboración propia.

Figura 4.

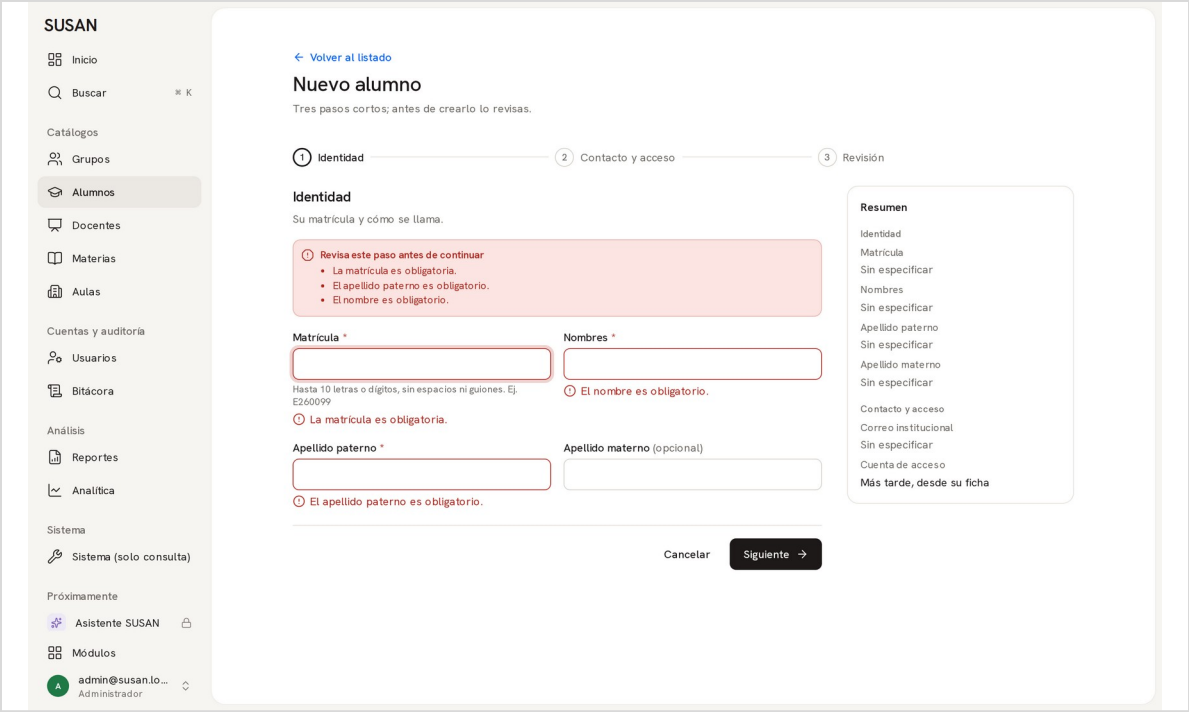
Pantalla de inicio de sesión en estado de envío incompleto con errores.



Nota. Cada error aparece junto a su campo con icono, texto y borde rojo; el botón no se apaga. Elaboración propia.

Figura 5.

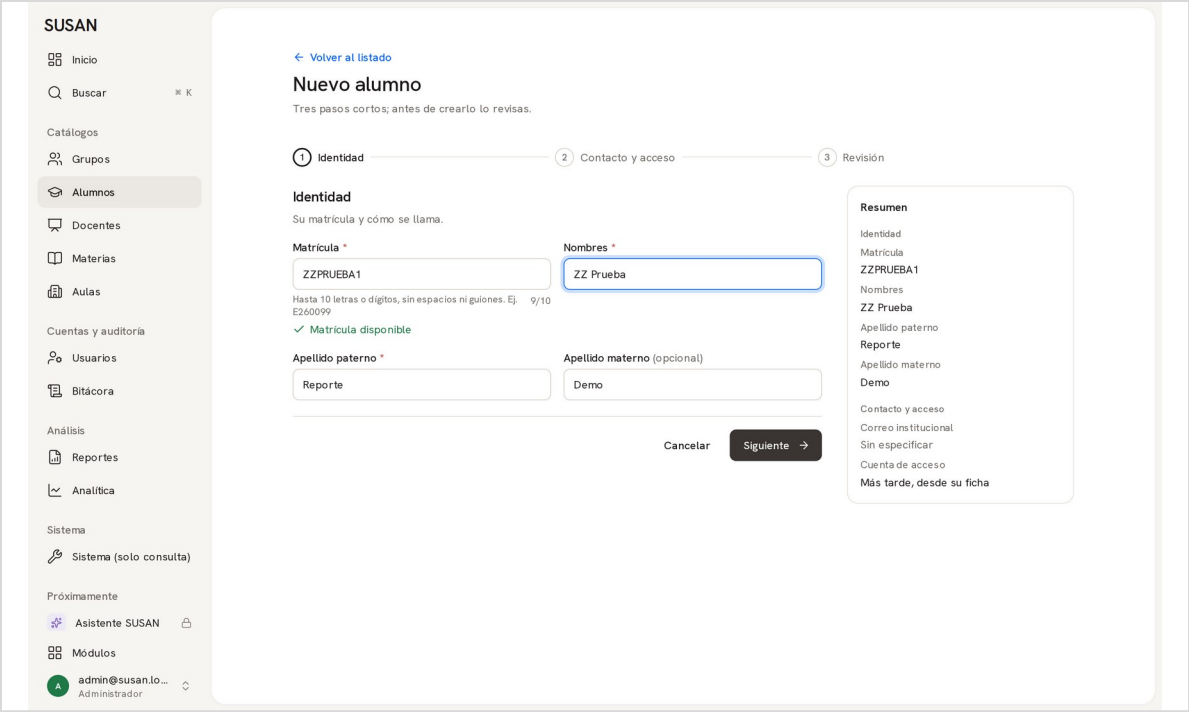
Pantalla de alta de alumno, paso 1 Identidad, en estado de errores al avanzar.



Nota. El resumen «Revisa este paso antes de continuar» enumera los errores y el foco pasa al primer campo inválido. Elaboración propia.

Figura 6.

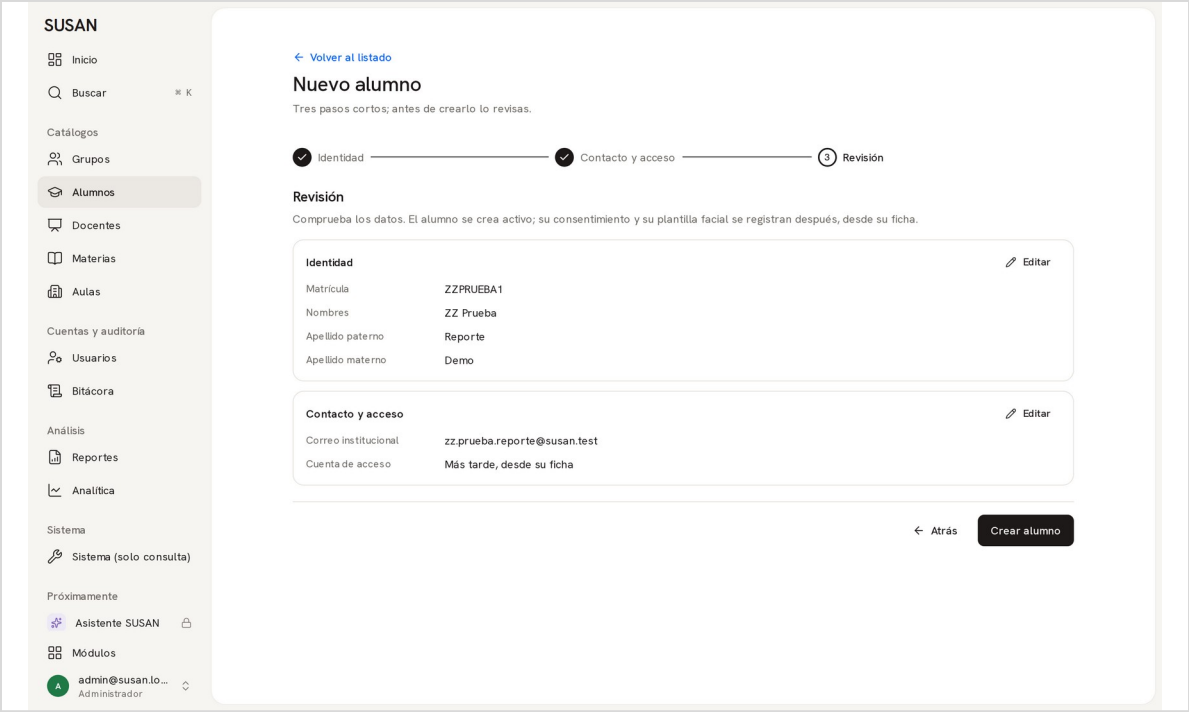
Pantalla de alta de alumno, paso 1 Identidad, en estado lleno y válido.



Nota. La matrícula se normaliza a mayúsculas, muestra el contador 9/10 y el mensaje «Matrícula disponible»; el resumen de la derecha se actualiza en vivo. Datos de prueba. Elaboración propia.

Figura 7.

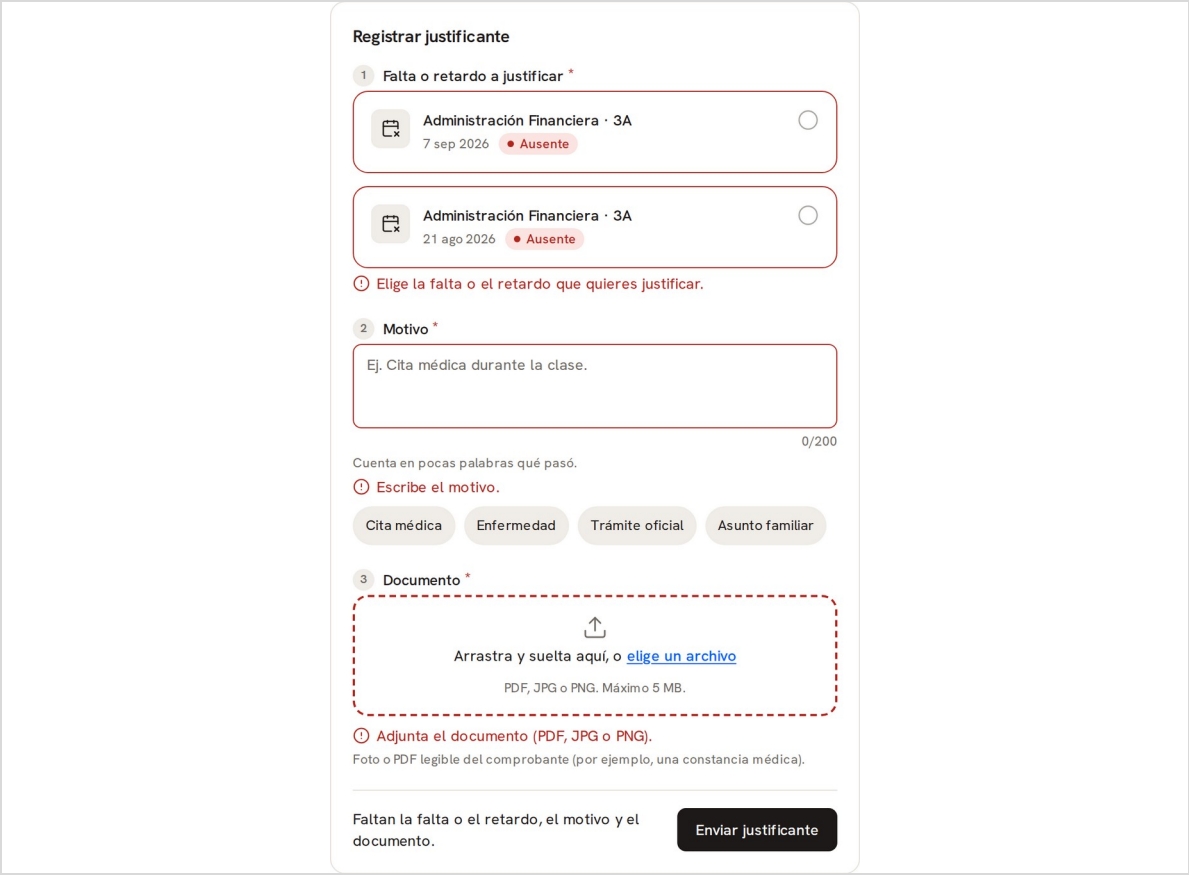
Pantalla de alta de alumno, paso 3 Revisión, en estado previo a la creación.



Nota. Cada sección tiene su botón «Editar» y la acción final es «Crear alumno» (verbo y objeto). Datos de prueba. Elaboración propia.

Figura 8.

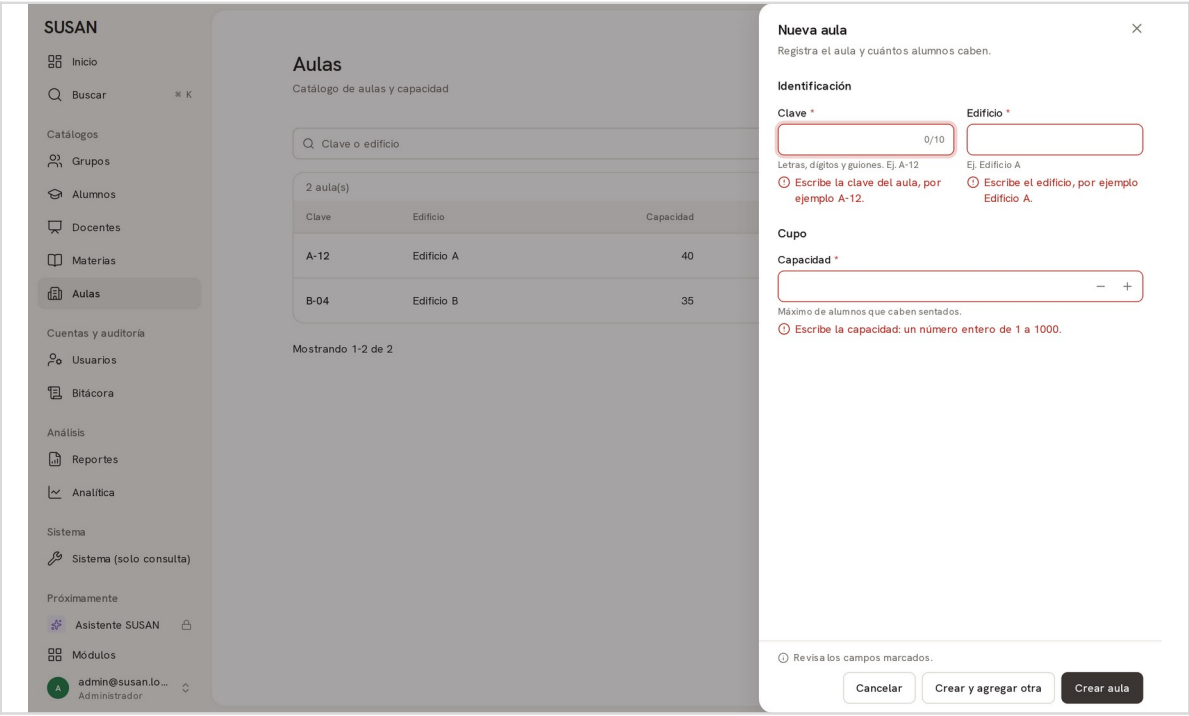
Pantalla de registro de justificante en estado de envío incompleto.



Nota. Tres bloques numerados con errores en cada uno; la barra inferior dice qué falta y el botón permanece activo. Recorte de la tarjeta del formulario. Elaboración propia.

Figura 9.

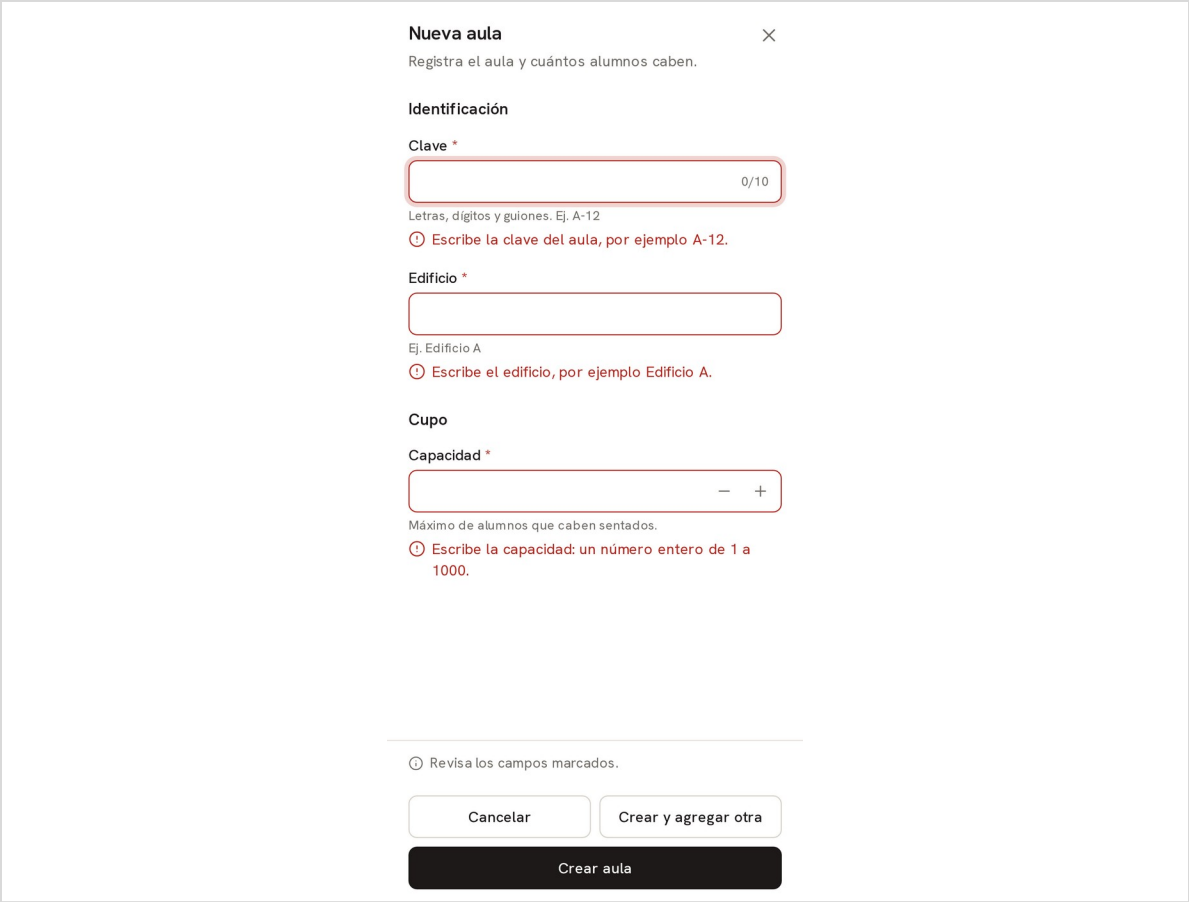
Pantalla de alta de aula en panel lateral en estado de envío vacío.



Nota. Ruta `/admin/aulas`; el pie fijo indica «Revisa los campos marcados.» y ofrece «Crear y agregar otra». Elaboración propia.

Figura 10.

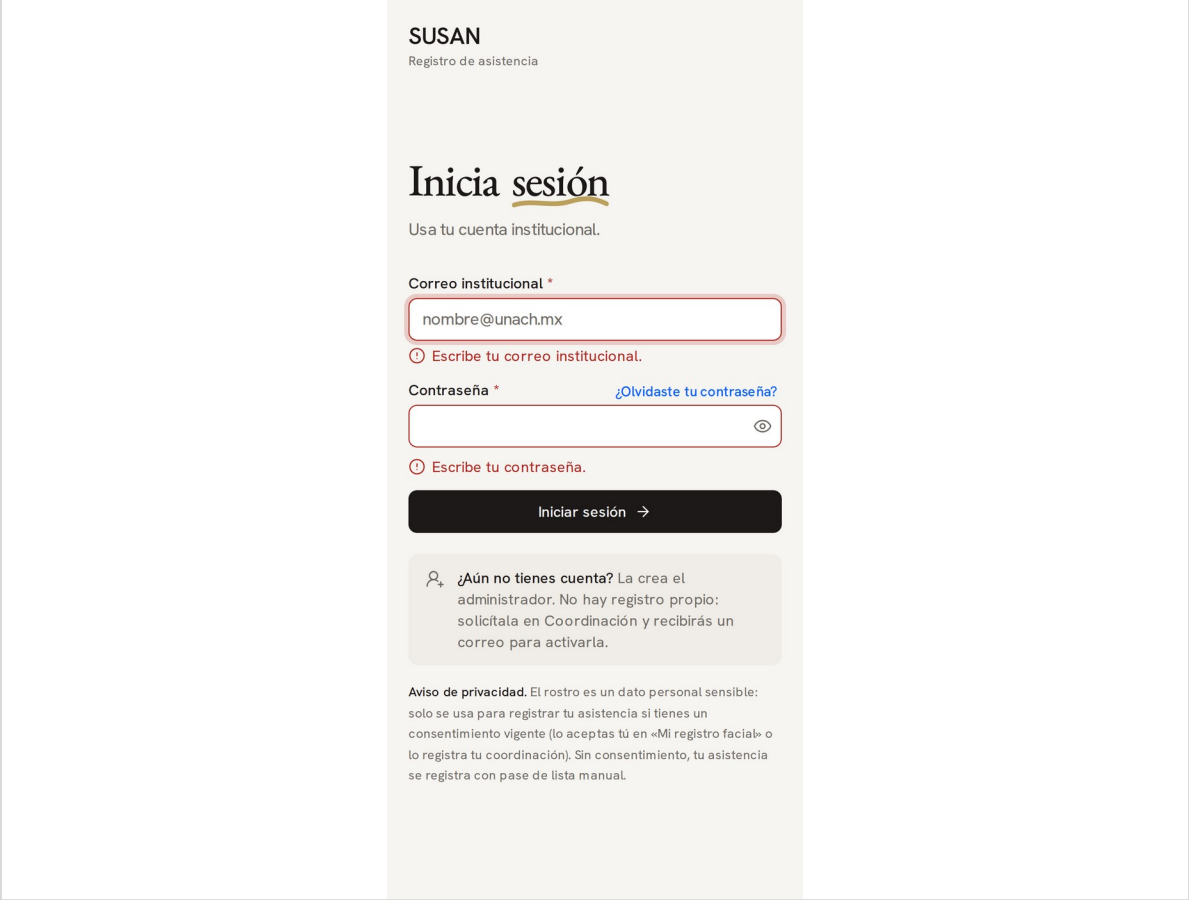
Pantalla móvil de alta de aula en estado de envío vacío.



Nota. En teléfono el panel lateral ocupa toda la pantalla (390x844). Elaboración propia.

Figura 11.

Pantalla móvil de inicio de sesión en estado de envío vacío con errores.



Nota. Los campos y el botón ocupan el ancho disponible sin desborde horizontal. Elaboración propia.

4. Organización del código.

Por convención del proyecto, cada componente vive en su propia carpeta y las páginas solo lo componen. La Tabla 2 enumera los diez archivos de `web/src` que se copiaron literalmente para este reporte (1345 líneas en total); cada copia se comparó con `diff -q` contra el archivo del repositorio y es idéntica.

Archivo	Ruta de origen (`web/src/`)	Líneas	Función
`Field.jsx`	`components/ui/Field/`	150	Primitiva de campo
`useFormulario.js`	`hooks/ui/`	223	Estado y validación

Archivo	Ruta de origen (`web/src/`)	Líneas	Función
`login.utils.js`	`components/auth/LoginForm/`	53	Reglas del acceso
`LoginForm.jsx`	`components/auth/LoginForm/`	76	Formulario de acceso
`alumnoForm.utils.js`	`components/admin/alumnos/AlumnoForm/`	118	Reglas del alumno
`CamposAlumno.jsx`	`components/admin/alumnos/AlumnoForm/`	136	Campos compartidos
`AlumnoForm.jsx`	`components/admin/alumnos/AlumnoForm/`	102	Ficha del alumno
`AltaAlumno.jsx`	`components/admin/alumnos/NuevoAlumno/`	182	Asistente de alta
`registrarJustificante.utils.js`	`components/alumno/RegistrarJustificanteForm/`	76	Reglas del justificante
`RegistrarJustificanteForm.jsx`	`components/alumno/RegistrarJustificanteForm/`	229	Formulario guiado

Nota. Tabla 2. Elaboración propia con base en `reporte/form/codigo/INDICE.md`. No se copiaron los contenedores `FormActions`, `FormSection`, `Asistente`, `Stepper`, `SuccessState`, `PanelLateral` e `Input`, ni `lib/formularios/limites.js`, que los archivos anteriores importan.

La división es la misma en todos los formularios: un componente de presentación (`.jsx`), un archivo de funciones puras con las reglas y los mensajes (`*.utils.js`, que se prueba por separado), el hook compartido `useFormulario` que gestiona valores, errores, foco y envío, y las llamadas al servidor, que pasan siempre por `@/lib/api/*` o `@/lib/http/*` y nunca directamente por Supabase.

5. Código de los formularios.

El texto completo de los diez archivos de la Tabla 2 se entrega en el archivo de anexos; aquí se reproducen tres fragmentos sin modificar.

La primitiva `Field` genera los identificadores, compone el texto de ayuda, el error y el estado «Válido», y los enlaza con el control mediante `aria-describedby`.

Código JavaScript.

```
    const generado = useId();
    const esGrupo = as === 'fieldset';
    const delegaMensajes = !esGrupo && children?.type?.recibeMensajes === true;
    const propsHijo = !esGrupo ? children?.props ?? {} : {};
    const id = htmlFor || propsHijo.id || generado;
    const hintId = hint || example ? `${id}-hint` : undefined;
    const errorId = error ? `${id}-error` : undefined;
    const validoTexto = valid && !error ? (valid === true ? 'Válido' :
String(valid)) : null;
    const validId = validoTexto ? `${id}-valid` : undefined;
    const describedby = [propsHijo['aria-describedby'], hintId, errorId,
validId].filter(Boolean).join(' ') || undefined;

    const etiqueta = (
        <>
            {label}
            {required && <span aria-hidden="true" className="text-destructive">
*</span>}
            {optional && !required && <span className="font-normal text-muted-
foreground"> (opcional)</span>}
        </>
    );

    const ayuda = (hint || example) && (
        <p id={hintId} className="text-xs text-muted-foreground">
            {hint}
            {hint && example ? ' ' : ''}
            {example && (YA_TRAE_PREFIJO.test(example) ? example : `Ej. $
{example}`)}
        </p>
    );

    const estado = error ? (
        <p id={errorId} role="alert" className="flex items-start gap-1.5 text-sm
text-destructive">
            <CircleAlert aria-hidden="true" className="mt-0.5 size-4 shrink-0"
strokeWidth={1.75} />
            {error}
        </p>
    ) : validoTexto ? (
        <p id={validId} role="status" className="flex items-center gap-1.5 text-sm
text-presente">
            <Check aria-hidden="true" className="size-4 shrink-0" strokeWidth={2} />
```

```
            {validoTexto}
        </p>
    ) : null;
    const ranura = reserveError ? <div className="min-h-5">{estado}</div> : estado;
```

La lógica de pasos del asistente de alta de alumno define cuáles campos valida cada paso y declara los pasos de captura; la revisión y el éxito los aporta `ui/Asistente`.

Código JavaScript.

```
    const campoPasos = (id) => f.validarCampos(CAMPOS_POR_PASO[id]) ||
f.erroresDe(CAMPOS_POR_PASO[id]);

    const pasos = [
        {
            id: 'identidad',
            titulo: 'Identidad',
            descripcion: 'Su matrícula y cómo se llama.',
            contenido: ({ irAPaso: ir }) => (
                <>
                    <GuardaIrAPaso irAPaso={ir} destinoRef={irAPaso} />
                    <div className="@container">
                        <div className="grid grid-cols-1 gap-x-4 gap-y-4 @md:grid-
cols-2">
                            <CamposIdentidad f={f} matricula={existencia} />
                        </div>
                    </div>
                </>
            ),
        },
        {
            id: 'contacto',
            titulo: 'Contacto y acceso',
            descripcion: 'Su correo institucional y, si quieres, su cuenta para
entrar.',
            contenido: ({ irAPaso: ir }) => (
                <>
                    <GuardaIrAPaso irAPaso={ir} destinoRef={irAPaso} />
                    <div className="@container flex flex-col gap-5">
                        <div className="grid grid-cols-1 gap-x-4 gap-y-4 @md:grid-
cols-2">
                            <CamposContacto f={f} />
                        </div>
                        <div className="flex flex-col gap-2">
```

```
                            <p className="text-sm font-medium">
                                Cuenta de acceso <span className="font-normal text-
muted-foreground">(opcional)</span>
                            </p>
                            <div className="rounded-xl border border-border p-4">
                                <Switch
                                    label="Crear su cuenta al terminar"
                                    hint="Con ella el alumno consulta su asistencia.
Se genera una contraseña temporal que verás una sola vez."
                                    checked={f.valores.crearCuenta}
                                    onChange={(valor) => f.setCampo('crearCuenta',
valor)}
                                    onLabel="Se creará"
                                    offLabel="Más tarde"
                                />
                            </div>
                        </div>
                    </div>
                </>
            ),
        },
    ];
```

Las reglas de validación del alumno, con los mensajes reales en español, y la función que arma el payload ya normalizado se mantienen en un archivo de funciones puras.

Código JavaScript.

```
/** Validación de conveniencia (la real es de la base). Devuelve { campo: mensaje }.
*/
export function validarAlumno(v) {
    const e = {};
    const matricula = mayusculas(recortar(v.matricula));
    if (!matricula) e.matricula = 'La matrícula es obligatoria.';
    else if (matricula.length > LIMITES.matricula) e.matricula = `La matrícula
admite hasta ${LIMITES.matricula} caracteres.`;
    else if (!PATRONES.matricula.test(matricula)) e.matricula = 'Usa solo letras y
dígitos, sin espacios ni guiones.';

    const paterno = recortar(v.apellido_paterno);
    if (!paterno) e.apellido_paterno = 'El apellido paterno es obligatorio.';
    else if (paterno.length > LIMITES.apellidos) e.apellido_paterno = `El apellido
paterno admite hasta ${LIMITES.apellidos} caracteres.`;
```

```
    if (recortar(v.apellido_materno).length > LIMITES.apellidos) e.apellido_materno
= `El apellido materno admite hasta ${LIMITES.apellidos} caracteres.`;

    const nombres = recortar(v.nombres);
    if (!nombres) e.nombres = 'El nombre es obligatorio.';
    else if (nombres.length > LIMITES.nombres) e.nombres = `El nombre admite hasta $
{LIMITES.nombres} caracteres.`;

    const correo = recortar(v.correo_institucional);
    if (!correo) e.correo_institucional = 'El correo institucional es obligatorio.';
    else if (!esCorreo(correo)) e.correo_institucional = 'Escribe un correo válido,
con la forma nombre@dominio.';
    return e;
}

/** Payload de api_alumno_guardar: apellidos y nombres siempre por separado y ya
normalizados. */
export function datosAlumno(v, idAlumno) {
    return {
        ...(idAlumno ? { id_alumno: idAlumno } : {}),
        matricula: mayusculas(recortar(v.matricula)),
        apellido_paterno: normalizarNombre(v.apellido_paterno),
        apellido_materno: normalizarNombre(v.apellido_materno) || null,
        nombres: normalizarNombre(v.nombres),
        correo_institucional: normalizarCorreo(v.correo_institucional),
    };
}
```

6. Estado de verificación.

Las pruebas automáticas se ejecutaron con `npx vitest run` sobre las carpetas de formularios y de primitivas (`auth`, `admin`, `alumno`, `ui`, `hooks/ui`, `lib/formularios`, `sesion/RegistroManualDialog`, `revision`, `tecnico`, `reportes` y `analitica/SelectorGrupo`). El resultado fue de **176 archivos de prueba y 2008 pruebas, de las cuales 2007 pasaron y 1 falló**, en 123 s con el servidor de desarrollo y Playwright corriendo a la vez. La prueba que falló es la de `DeteccionRostros.test.jsx` («carga el detector, dibuja un recuadro por rostro y avisa cuántos ve»: esperaba el estado `listo` y recibió `cargando`), un temporizado ajeno a los formularios; al repetirla sola (`npx vitest run src/components/ui/DeteccionRostros`) pasaron 2 archivos y 19 pruebas. `docs/design/formularios/PRIMITIVAS-campos.md` registra el mismo caso. Todos los formularios tienen archivo de prueba propio, salvo `ConsolaTecnico`, que solo compone pestañas, y `ParametroFila` y `CampoNumerico`, que tienen únicamente la prueba de sus utilidades.

Las 114 capturas se revisaron una por una con la herramienta de lectura; se repitieron las que salieron mal (estado persistente de un error previo, formularios cortados por la ventana, sesión sin hidratar). Muestran los estados vacío, con errores, válido y de éxito de la mayoría de los formularios, la revisión previa a crear en los asistentes y cinco pantallas móviles.

Lo que **no** queda cubierto es lo siguiente:

- No se corrió la suite completa (`web/src` tiene 296 archivos `*.test.*`), ni `npm run lint` ni `npm run build`.
- El registro facial completo no se capturó: Chromium no tiene cámara real y el reconocimiento usó el embedder `fake`; se llegó hasta la pantalla de toma.
- No se ejecutaron el pase de lista manual ni el cierre de sesión, y no se confirmaron la importación CSV, el guardado de parámetros, el estatus de cámaras ni la reconstrucción del índice, para no alterar los datos de demostración.
- Todas las capturas son en tema claro (la interfaz no tiene interruptor de modo oscuro), y el aviso de Bloq Mayús y la protección `beforeunload` solo tienen pruebas de componente, no se ejercitaron en navegador.
- El diseño no se cotejó pantalla por pantalla contra las pantallas P-01 a P-09 de `docs/spec/Avance1.txt`.
- Las capturas con éxito ejecutaron operaciones reales contra la base local; quedó un residuo en la bitácora de solo anexar (una cuenta de prueba desactivada y su alumno), por lo que los conteos de la base ya no coinciden con los de la semilla.

ESTRUCTURA DE LA BASE DE DATOS.

La base de datos de SUSAN es el núcleo del sistema: ahí se consolida el registro de asistencia y ahí se aplican las reglas de negocio. Se implementó en PostgreSQL 16 y, para este reporte, sus cifras se obtuvieron consultando el catálogo de la base viva (`pg_class`, `pg_constraint`, `pg_proc`, `pg_policies`, entre otros) y las migraciones del commit `f742bb4`, y no se tomaron de la documentación. El modelo actual tiene **20 tablas** y una vista (`v_resumen_asistencia`) en el esquema `public`, con 135 columnas, 20 llaves primarias, **26 llaves foráneas**, 21 restricciones `UNIQUE`, 35 restricciones `CHECK`, 14 tipos enumerados y 64 índices. La lógica vive en **98 funciones** de `public` (76 `api_*`, que llama la interfaz, y 22 `svc_*`, reservadas al servicio de reconocimiento y a los trabajos programados), 87 funciones internas del esquema `app` y 7 disparadores. Las 20 tablas tienen seguridad a nivel de fila (RLS) con **39 políticas**, todas de lectura. El esquema se construye con **43 migraciones** (43 de 43 aplicadas en la base verificada) y se prueba con **37 archivos de pruebas SQL** que suman 2 461 casos (1 508 positivos y 953 negativos).

La decisión de diseño central es que **la lógica de negocio reside en PostgreSQL** (funciones RPC, disparadores y restricciones) y no en la interfaz: esta solo presenta datos y

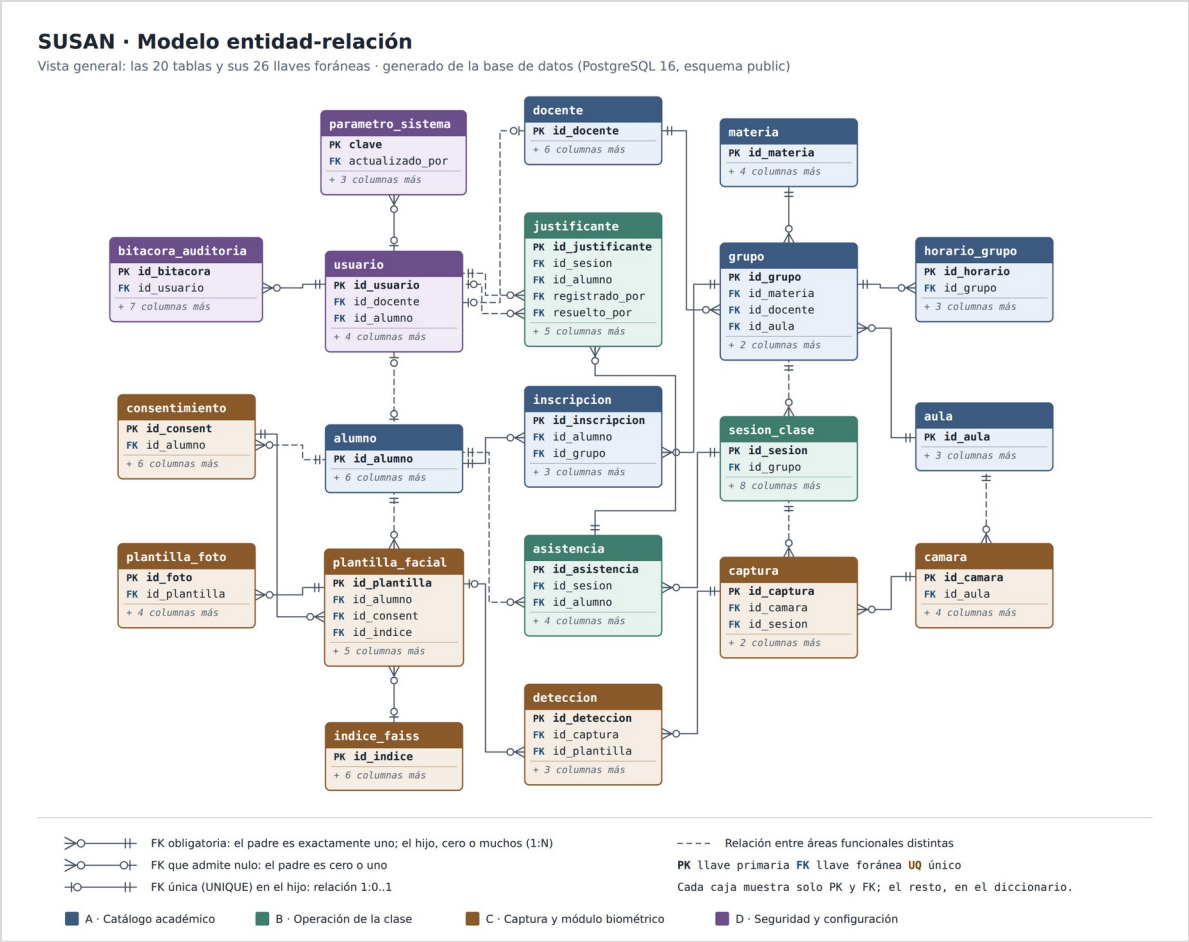
captura decisiones, y no consulta tablas directamente. Las funciones de escritura son `SECURITY DEFINER` y comprueban el rol y la pertenencia al inicio; las lecturas son en su mayoría `SECURITY INVOKER`, de modo que la RLS decide qué filas ve cada usuario. Los roles `anon` y `authenticated` no tienen privilegios de escritura sobre las tablas (0 concesiones de `anon` y 0 de `authenticated` distintas de `SELECT`), y por el requisito RNF-03 ninguna de las 135 columnas es binaria ni de arreglo: la biometría se limita a identificadores del índice y a rutas de objetos.

1. Modelo entidad-relación.

La figura 1 presenta el modelo completo, con las 20 tablas coloreadas por módulo; en ella cada caja muestra solo las llaves y las demás columnas se resumen. Las figuras siguientes detallan cada módulo con todas las columnas y sus tipos.

Figura 12.

Modelo entidad-relación — vista general.

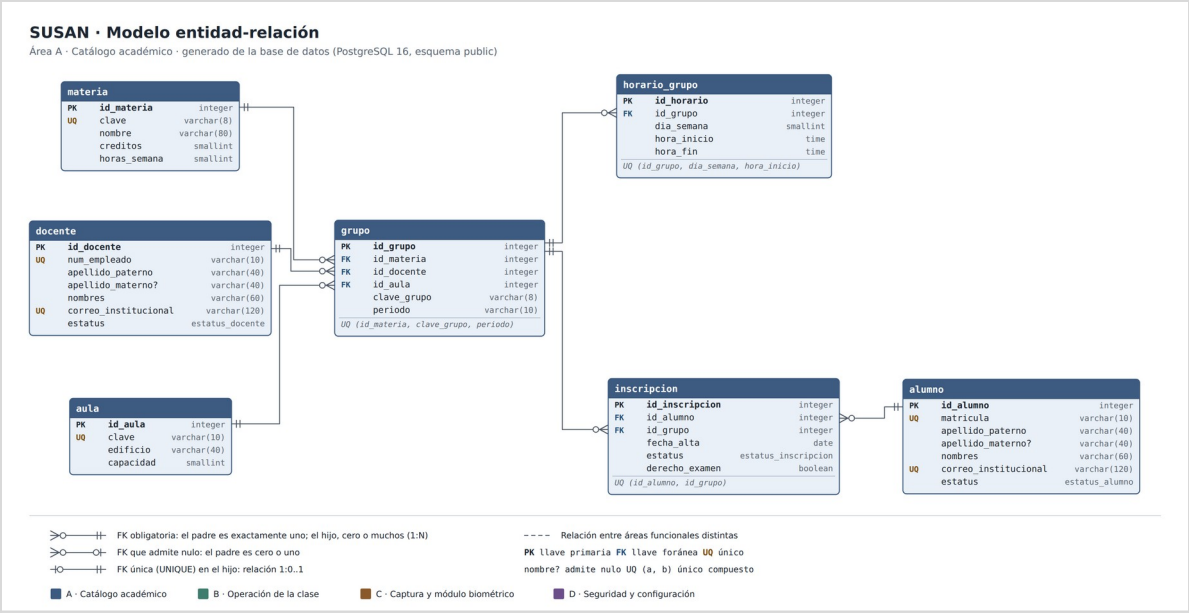


Nota. Elaboración propia, generado a partir del catálogo de la base de datos. Azul: catálogo académico; verde: operación de la clase; café: captura y biometría; morado: seguridad y configuración. Las líneas discontinuas unen tablas de módulos distintos.

Para leer los diagramas, la línea con pata de gallo indica el lado «muchos» de una relación 1:N, un círculo indica que el lado es opcional, y PK, FK y UQ marcan llaves primarias, foráneas y restricciones únicas. Un signo de interrogación junto al nombre de una columna indica que admite nulo. Las cajas grises de borde discontinuo son tablas de otro módulo que se repiten solo con su llave para mostrar la conexión.

Figura 13.

Modelo entidad-relación — módulo catálogo académico.

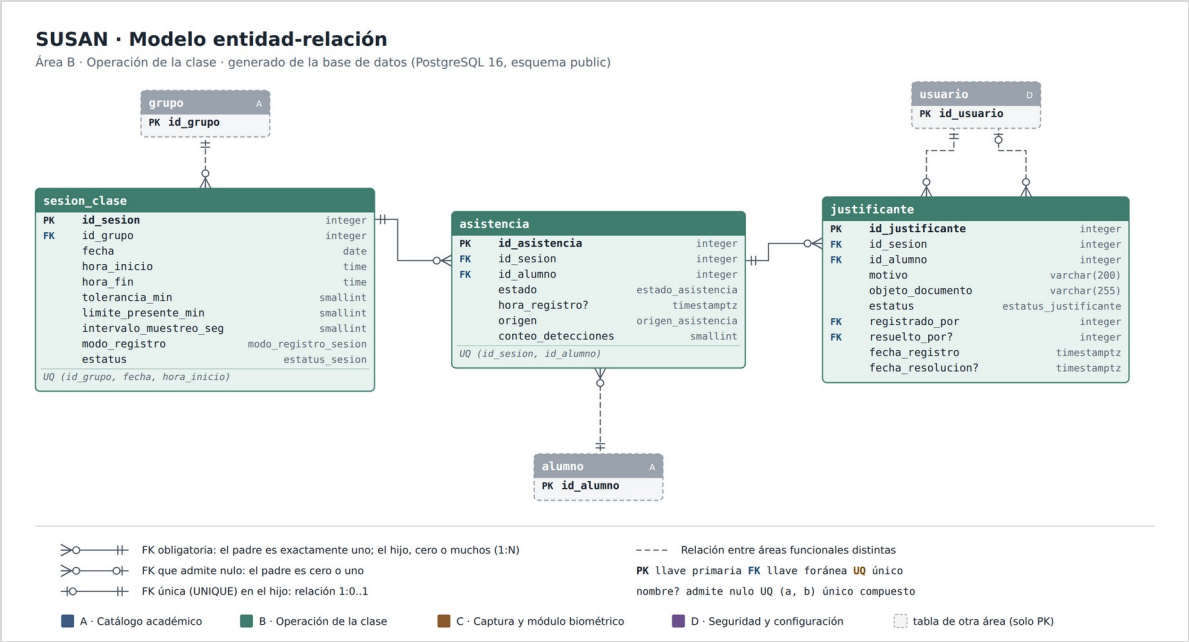


Nota. Elaboración propia. `grupo` une materia, docente y aula; `inscripcion` une alumno y grupo.

El módulo académico responde quién enseña qué, dónde y cuándo. El `grupo` es la pieza central: referencia a una materia, un docente y un aula, y de él cuelgan sus bloques de `horario_grupo` y las `inscripcion` de los alumnos.

Figura 14.

Modelo entidad-relación — módulo operación de la clase.

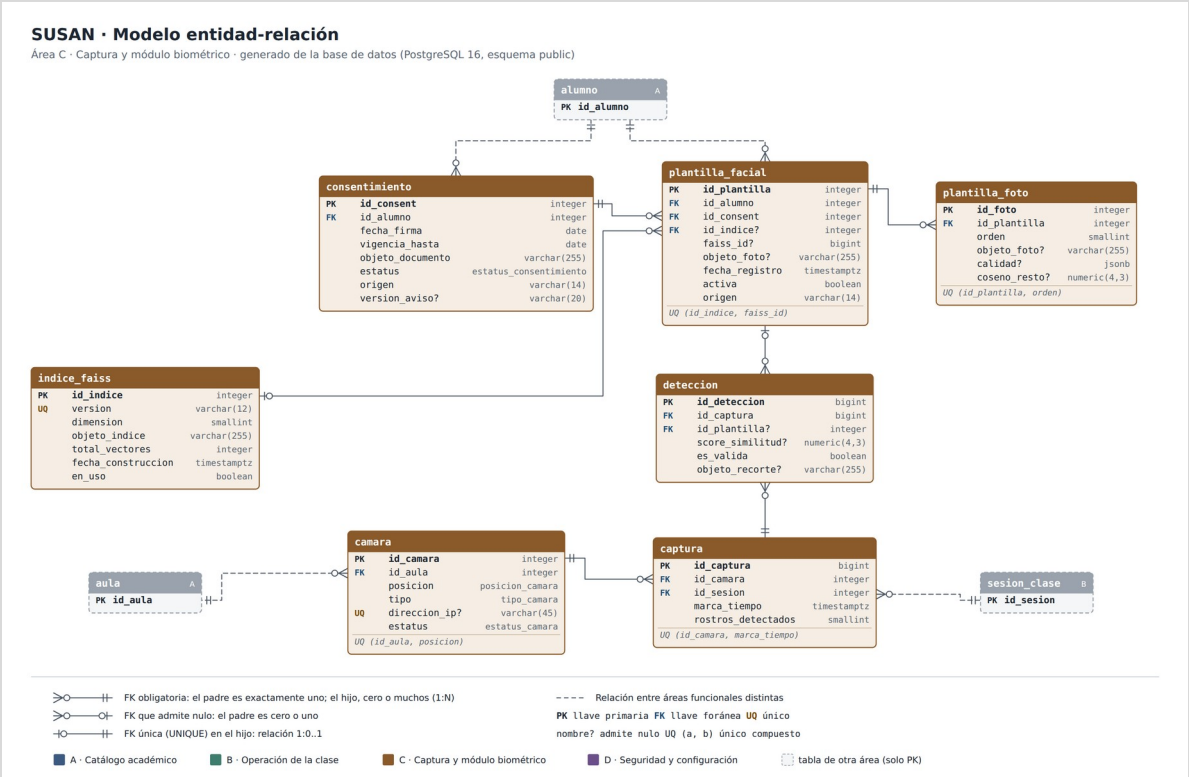


Nota. Elaboración propia. `justificante` se une a `asistencia` mediante la llave compuesta (id_sesion, id_alumno).

La tabla `asistencia` es el eje del modelo: guarda una fila por alumno y sesión, nace de una `sesion_clase` y puede tener justificantes. Como el justificante referencia la asistencia con llave compuesta, solo se justifica un registro que existe.

Figura 15.

Modelo entidad-relación — módulo captura y biometría.

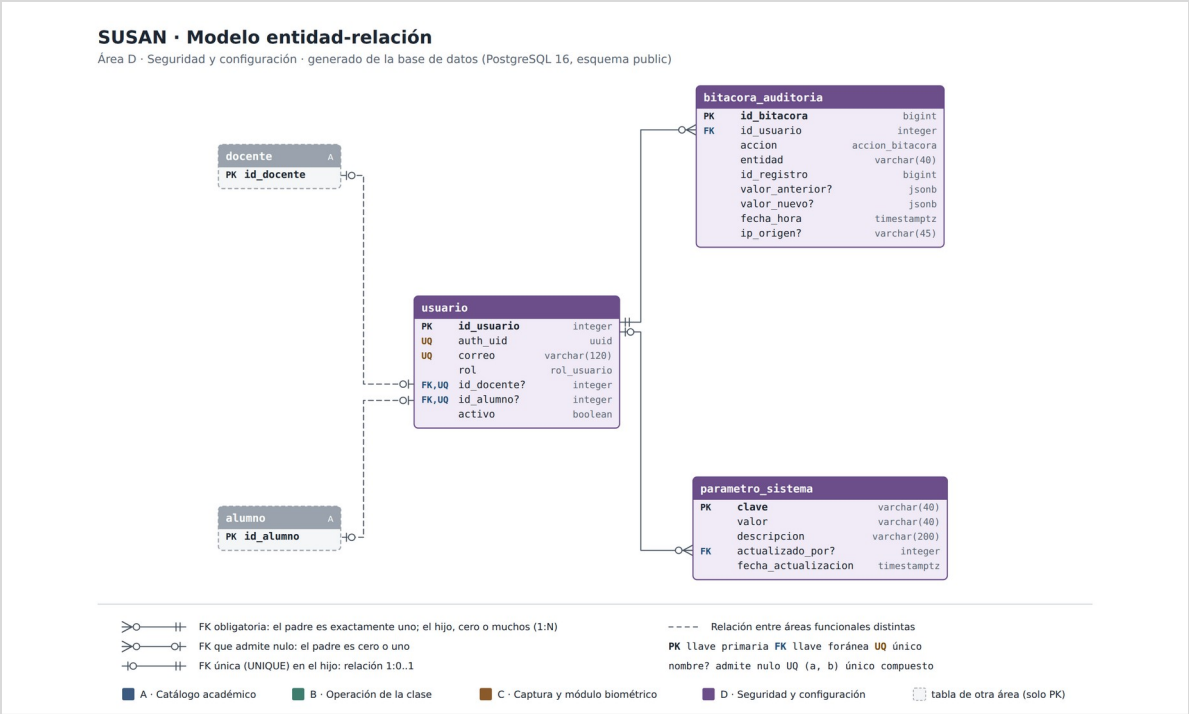


Nota. Elaboración propia. Ninguna tabla almacena vectores ni imágenes; solo rutas de objetos (`objeto_*`) e identificadores del índice.

El módulo biométrico enlaza el consentimiento del alumno con su plantilla facial y sus fotos, la versión del índice FAISS en uso, y la cadena cámara, captura y detección que alimenta la asistencia automática. La plantilla exige un consentimiento propio (`id_consent`), lo que sostiene la regla RN-13.

Figura 16.

Modelo entidad-relación — módulo seguridad y configuración.



Nota. Elaboración propia. `usuario` se vincula a Supabase Auth mediante `auth_uid`; la contraseña nunca se guarda en esta base.

El módulo de seguridad concentra las cuentas (`usuario`, ligada a un docente o a un alumno según su rol), los parámetros calibrables del sistema y la bitácora de auditoría, que registra quién hizo qué y con qué valor anterior y nuevo.

2. Tablas.

Las tablas se agrupan por módulo. Se listan los campos principales y las llaves y restricciones con sus nombres reales del catálogo; el diccionario completo (tipos, nulos, valores por omisión, índices y políticas) está en el archivo de anexos.

Catálogo académico. Siete tablas con los datos maestros de personas, materias, aulas y grupos.

Tabla	Campos principales	Llaves y restricciones
`alumno`	matrícula, nombres, apellidos, correo, estatus	PK id_alumno; UQ matrícula y correo
`docente`	núm. de empleado, nombres, apellidos, correo, estatus	PK id_docente; UQ núm. de empleado y correo
`materia`	clave, nombre, créditos, horas por semana	PK id_materia; UQ clave; CHECK créditos > 0
`aula`	clave, edificio, capacidad	PK id_aula; UQ clave; CHECK capacidad > 0
`grupo`	clave de grupo, periodo, materia, docente, aula	PK id_grupo; 3 FK; UQ (materia, grupo, periodo)
`horario_grupo`	día de la semana, hora de inicio y fin	PK id_horario; FK a grupo (CASCADE); CHECK de día y horas
`inscripcion`	alumno, grupo, fecha de alta, estatus, derecho a examen	PK id_inscripcion; 2 FK; UQ (alumno, grupo)

Nota. Elaboración propia a partir del diccionario de datos generado del catálogo. FK: llave foránea; UQ: restricción única; todas las FK son `ON DELETE RESTRICT` salvo la indicada.

Operación de la clase. Tres tablas que registran lo que ocurre en cada sesión.

Tabla	Campos principales	Llaves y restricciones
`sesion_clase`	grupo, fecha, horas, tolerancia, límite de presente, modo, estatus	PK id_sesion; FK grupo; UQ (grupo, fecha, hora); 3 CHECK
`asistencia`	sesión, alumno, estado, hora de registro, origen, conteo de lecturas	PK id_asistencia; 2 FK; UQ (sesión, alumno); 4 CHECK
`justificante`	motivo, documento, estatus, registró, resolvió, fechas	PK id_justificante; FK compuesta a asistencia; 2 FK a usuario; 2 CHECK

Nota. Elaboración propia. La vista `v_resumen_asistencia` calcula faltas equivalentes y porcentaje en cada consulta; no es una tabla.

Captura y biometría. Siete tablas; solo guardan rutas y valores numéricos, nunca imágenes ni vectores.

Tabla	Campos principales	Llaves y restricciones
`consentimiento`	alumno, fecha de firma, vigencia, documento, estatus, origen	PK id_consent; FK alumno; 2 CHECK (origen, vigencia)
`indice_faiss`	versión, dimensión, objeto del índice, total de vectores, en uso	PK id_indice; UQ versión; índice único parcial de una sola versión en uso
`plantilla_facial`	alumno, consentimiento, índice, `faiss_id`, activa, origen	PK id_plantilla; 3 FK; UQ (índice, faiss_id); 3 CHECK
`plantilla_foto`	plantilla, orden, ruta de la foto, calidad, coseno	PK id_foto; FK plantilla; UQ (plantilla, orden); 3 CHECK
`camara`	aula, posición, tipo, dirección IP, estatus	PK id_camara; FK aula; UQ (aula, posición) y UQ de IP; CHECK de IP
`captura`	cámara, sesión, marca de tiempo, rostros detectados	PK id_captura; 2 FK; UQ (cámara, marca); CHECK de rostros
`deteccion`	captura, plantilla, score de similitud, es válida, recorte	PK id_deteccion; 2 FK; CHECK de score y de candidato

Nota. Elaboración propia. `plantilla_foto` es la única tabla que no figuraba en la especificación del Avance 1.

Seguridad y configuración. Tres tablas de cuentas, parámetros y auditoría.

Tabla	Campos principales	Llaves y restricciones
`usuario`	`auth_uid`, correo, rol, docente o alumno asociado, activo	PK id_usuario; UQ de auth_uid, correo, docente y alumno; 2 CHECK
`bitacora_auditoria`	usuario, acción, entidad, registro, valor anterior y nuevo, fecha, IP	PK id_bitacora; FK usuario; CHECK de entidad y de valores
`parametro_sistema`	clave, valor, descripción, actualizado por, fecha	PK clave; FK usuario; 12 parámetros calibrables
Internas del esquema `app`	`bloqueo` y `reloj_demo` (2 tablas)	No se exponen por la API; apoyan arrendamientos y el reloj de demostración

Nota. Elaboración propia. Con estas tablas y las anteriores se completan las 20 tablas de `public`; las dos de `app` son internas.

3. Reglas de negocio en la base de datos.

Las reglas de la especificación se aplican en la base, de modo que ningún cliente puede saltárselas. Cada nombre de la tabla se comprobó contra el catálogo de la base viva. Los rechazos de los disparadores empiezan con el nombre de la regla y devuelven el código `23514`, igual que una restricción `CHECK`.

Regla	Mecanismo	Nombre real
RN-01 Solo con inscripción activa	Disparador	`trg_asistencia_rn01`; error `rn01_asistencia_inscripcion_activa`
RN-02 Un registro por alumno y sesión	Restricción única	`uq_asistencia_sesion_alumno`
RN-03 Presente o retardo según el minuto	Función y CHECK	`app.estado_por_minuto`; `ck_sesion_ventana`
RN-07 Cierre: inscritos sin registro quedan ausentes	Funciones	`app.cerrar_sesion`; `public.api_cerrar_sesion`
RN-08 Corrección dentro de 48 h con bitácora	Función y parámetro	`app.limite_correccion`; `horas_limite_correccion`

Regla	Mecanismo	Nombre real
RN-12 Una plantilla activa por alumno	Índices únicos parciales	`ux_plantilla_facial_activa`; `ux_indice_faiss_en_uso`
RN-13 Sin plantilla sin consentimiento vigente	Disparador	`trg_plantilla_rn13`; error `rn13_plantilla_consentimiento_vencido`
RNF-08 Bitácora de solo inserción	Disparadores y política RLS	`trg_bitacora_sin_cambios`; `trg_bitacora_sin_truncate`

Nota. Elaboración propia a partir del catálogo de la base. Los umbrales (por ejemplo `limite_presente_min` o `tolerancia_min`) viven en `parametro_sistema` y no en el código; la sesión los copia al programarse, por lo que un ajuste posterior no altera las sesiones ya registradas.

4. Código de base de datos.

Los fragmentos siguientes son copias literales de las migraciones del commit `f742bb4`; no son ejecutables por separado porque dependen de objetos creados en otras migraciones. El DDL completo y los extractos adicionales se incluyen en el archivo de anexos.

La tabla `asistencia` es el eje del modelo. Sus restricciones codifican varias reglas: una fila por alumno y sesión (RN-02), un pendiente solo si es automático y tiene lecturas (RN-05) y un estado de presencia que exige hora salvo corrección (RN-03).

Código SQL.

```
CREATE TABLE public.asistencia (
    id_asistencia       integer GENERATED ALWAYS AS IDENTITY,
    id_sesion           integer NOT NULL,
    id_alumno           integer NOT NULL,
    estado              public.estado_asistencia NOT NULL,
    hora_registro       timestamptz,
    origen              public.origen_asistencia NOT NULL,
    conteo_detecciones  smallint NOT NULL DEFAULT 0,
    CONSTRAINT pk_asistencia PRIMARY KEY (id_asistencia),
```

```
    CONSTRAINT fk_asistencia_sesion FOREIGN KEY (id_sesion) REFERENCES
public.sesion_clase (id_sesion) ON DELETE RESTRICT,
    CONSTRAINT fk_asistencia_alumno FOREIGN KEY (id_alumno) REFERENCES public.alumno
(id_alumno) ON DELETE RESTRICT,
    -- RN-02: un solo registro por alumno y sesion, sin importar cuantas detecciones
haya.
    CONSTRAINT uq_asistencia_sesion_alumno UNIQUE (id_sesion, id_alumno),
    CONSTRAINT ck_asistencia_conteo CHECK (conteo_detecciones >= 0),
    CONSTRAINT ck_asistencia_ausente_sin_hora CHECK (estado <> 'ausente' OR
hora_registro IS NULL),
    -- Un pendiente es automatico, tiene hora y al menos una lectura (RN-05).
    CONSTRAINT ck_asistencia_pendiente CHECK (
        estado <> 'pendiente'
        OR (origen = 'automatico' AND hora_registro IS NOT NULL AND
conteo_detecciones > 0)
    ),
    -- La hora decide el estado (RN-03); solo una correccion puede dejar
presente/retardo sin hora.
    CONSTRAINT ck_asistencia_hora_presencia CHECK (
        estado NOT IN ('presente', 'retardo') OR origen = 'correccion' OR
hora_registro IS NOT NULL
    )
);
COMMENT ON TABLE public.asistencia IS 'Resultado unico de un alumno en una sesion.
Triggers: trg_asistencia_rn01, trg_asistencia_ciclo.';
```

El disparador de la regla RN-01 se ejecuta antes de insertar una asistencia, exige una inscripción activa del alumno en el grupo de la sesión y la bloquea hasta el fin de la transacción para que una baja simultánea no se cuele.

Código SQL.

```
--
-------------------------------------------------------------------------------
---------
-- RN-01: antes de insertar, o de cambiar id_sesion / id_alumno, exige una
inscripcion ACTIVA del
-- alumno en el grupo de la sesion y la bloquea (FOR SHARE) hasta el fin de la
transaccion, para
-- que una baja simultanea no se cuele entre la comprobacion y el commit.
--
-------------------------------------------------------------------------------
---------
CREATE FUNCTION app.trg_asistencia_rn01() RETURNS trigger
```

```
    LANGUAGE plpgsql SECURITY DEFINER
    SET search_path = public, app, pg_temp
AS $$
DECLARE
    v_id_inscripcion integer;
BEGIN
    SELECT i.id_inscripcion INTO v_id_inscripcion
      FROM public.inscripcion i
      JOIN public.sesion_clase s ON s.id_grupo = i.id_grupo
     WHERE s.id_sesion = NEW.id_sesion
       AND i.id_alumno = NEW.id_alumno
       AND i.estatus = 'activa'
       FOR SHARE OF i;
    IF NOT FOUND THEN
        PERFORM app.rechazar_regla('rn01_asistencia_inscripcion_activa',
            'el alumno no tiene una inscripción activa en el grupo de la sesión');
    END IF;
    RETURN NEW;
END
$$;

CREATE TRIGGER trg_asistencia_rn01
    BEFORE INSERT OR UPDATE OF id_sesion, id_alumno ON public.asistencia
    FOR EACH ROW EXECUTE FUNCTION app.trg_asistencia_rn01();
```

La función `app.estado_por_minuto` implementa la ventana de RN-03: presente antes del límite, retardo hasta la tolerancia y nulo fuera de ella.

Código SQL.

```
CREATE OR REPLACE FUNCTION app.estado_por_minuto(p_limite integer, p_tolerancia
integer, p_minuto numeric)
    RETURNS public.estado_asistencia
    LANGUAGE sql IMMUTABLE
    SET search_path = pg_catalog, pg_temp
AS $$
    SELECT CASE WHEN p_minuto < p_limite THEN 'presente'::public.estado_asistencia
                WHEN p_minuto <= p_tolerancia THEN
'retardo'::public.estado_asistencia
           END
$$;
COMMENT ON FUNCTION app.estado_por_minuto(integer, integer, numeric) IS
    'RN-03: presente si minuto < limite, retardo si limite <= minuto <= tolerancia,
NULL fuera de la ventana.';
```

La RPC `api_cerrar_sesion` es la puerta que usa la interfaz: comprueba el rol docente, bloquea la sesión y delega el cierre (RN-07) en la función interna.

Código SQL.

```
CREATE OR REPLACE FUNCTION public.api_cerrar_sesion(p_id_sesion integer) RETURNS
jsonb
    LANGUAGE plpgsql VOLATILE SECURITY DEFINER
    SET search_path = public, app, pg_temp
AS $$
DECLARE
    s public.sesion_clase%ROWTYPE;
BEGIN
    PERFORM app.exigir_rol('docente');
    s := app.sesion_bloqueada(p_id_sesion, false);
    RETURN app.cerrar_sesion(s.id_sesion, true);
END
$$;
```

Las políticas de seguridad a nivel de fila de `asistencia` definen quién lee qué: el administrador todo, el docente solo los grupos que le corresponden y el alumno únicamente lo propio.

Código SQL.

```
-- asistencia: la politica de ejemplo de la spec (docente_ve_sus_grupos), mas admin
(reportes de
-- todos los grupos) y alumno (historial propio, RF-21).
CREATE POLICY admin_ve_todo ON public.asistencia FOR SELECT TO authenticated
    USING ((SELECT app.es_admin()));
CREATE POLICY docente_ve_sus_grupos ON public.asistencia FOR SELECT TO authenticated
    USING ((SELECT app.rol()) = 'docente'
           AND EXISTS (SELECT 1 FROM public.sesion_clase s
                       WHERE s.id_sesion = asistencia.id_sesion
                         AND s.id_grupo IN (SELECT app.grupos_visibles()))));
CREATE POLICY alumno_ve_lo_propio ON public.asistencia FOR SELECT TO authenticated
    USING (id_alumno = (SELECT app.id_alumno()));
```

Por último, los disparadores de la bitácora impiden modificar, borrar o vaciar la tabla de auditoría (RNF-08), incluso durante cargas masivas.

Código SQL.

```
--
-------------------------------------------------------------------------------
---------
-- RNF-08: la bitacora es de solo insercion. ENABLE ALWAYS: tambien protege con
-- session_replication_role = replica (el modo que usa la carga masiva de la
semilla).
--
-------------------------------------------------------------------------------
---------
CREATE FUNCTION app.trg_bitacora_sin_cambios() RETURNS trigger
    LANGUAGE plpgsql
    SET search_path = pg_catalog, pg_temp
AS $$
BEGIN
    RAISE EXCEPTION 'trg_bitacora_sin_cambios: la bitácora de auditoría es de solo
inserción (%)', TG_OP
        USING ERRCODE = 'integrity_constraint_violation', CONSTRAINT =
'trg_bitacora_sin_cambios';
END
$$;

CREATE FUNCTION app.trg_bitacora_sin_truncate() RETURNS trigger
    LANGUAGE plpgsql
    SET search_path = pg_catalog, pg_temp
AS $$
BEGIN
    RAISE EXCEPTION 'trg_bitacora_sin_truncate: la bitácora de auditoría no admite
TRUNCATE'
        USING ERRCODE = 'integrity_constraint_violation', CONSTRAINT =
'trg_bitacora_sin_truncate';
END
$$;

CREATE TRIGGER trg_bitacora_sin_cambios
    BEFORE UPDATE OR DELETE ON public.bitacora_auditoria
    FOR EACH ROW EXECUTE FUNCTION app.trg_bitacora_sin_cambios();
CREATE TRIGGER trg_bitacora_sin_truncate
    BEFORE TRUNCATE ON public.bitacora_auditoria
    FOR EACH STATEMENT EXECUTE FUNCTION app.trg_bitacora_sin_truncate();
```

```
ALTER TABLE public.bitacora_auditoria ENABLE ALWAYS TRIGGER
trg_bitacora_sin_cambios;
ALTER TABLE public.bitacora_auditoria ENABLE ALWAYS TRIGGER
trg_bitacora_sin_truncate;
```

5. Estado de verificación.

Pruebas SQL. El 30 de octubre de 2026 se ejecutó `scripts/test-sql.sh` sobre el commit `f742bb4` contra PostgreSQL 16 local, con una base temporal que el propio script crea y elimina. El resultado fue de **37 archivos, 0 con fallas, 1 508 pruebas positivas y 953 negativas, 0 fallos** (13.2 s). Las positivas comprueban que lo permitido funciona; las negativas, que la base rechaza lo prohibido con la restricción o el código de error esperado.

Cotejo contra la base viva. Las cifras de este apartado salen de consultas al catálogo de la base `susan` (PostgreSQL 16.14, 43 de 43 migraciones registradas). Además: (a) los 12 extractos del archivo de código resultaron idénticos, byte a byte (21 174 bytes), a sus rangos de líneas en `supabase/migrations/`; (b) el cuerpo de las 6 funciones citadas en esos extractos coincide con el que corre en la base (`pg_proc.prosrc`, una sola versión de cada una); y (c) todos los nombres de restricciones, disparadores, funciones, índices y parámetros de la tabla de reglas existen en el catálogo.

Diferencias frente a la especificación. La especificación del Avance 1 describe 19 tablas y 125 columnas; la base actual tiene 20 tablas y 135 columnas. La comparación columna por columna arrojó 0 columnas de la especificación ausentes en la base; las diferencias son ampliaciones:

- Una tabla nueva, `plantilla_foto` (6 columnas), con su llave primaria, llave foránea, restricción única, tres `CHECK` y un disparador.
- Cuatro columnas nuevas en tablas existentes: `camara.tipo`, `consentimiento.origen`, `consentimiento.version_aviso` y `plantilla_facial.origen`.
- 7 disparadores en lugar de 6, 14 tipos enumerados en lugar de 13 y 12 parámetros del sistema en lugar de 11. El tipo `accion_bitacora` tiene 10 valores: los 8 de la especificación más las altas y revocaciones de consentimiento.
- `objeto_foto` admite nulo en `plantilla_facial` y `plantilla_foto` (en la especificación era obligatorio): cuando el vector ya está en el índice en uso, la foto de registro se descarta.
- Las reglas que dependen de la hora o del usuario autenticado (apertura en horario, plazo de 48 h) viven dentro de funciones y no como restricciones `CHECK`.
- La lista de entidades auditables de la bitácora (`ck_bitacora_entidad`, 18 entidades) no incluye `plantilla_foto`.

Limitaciones. La base compartida contiene algunas filas de prueba con prefijo zz que no pertenecen a la semilla y no se borraron; no afectan las cifras de esquema. Las pruebas SQL se ejecutaron sobre una base temporal y no sobre la base compartida. La integración entre la estación de aula con tarjeta RFID y estas funciones aún no existe, por lo que no forma parte de lo verificado aquí.

El DDL completo generado con `pg_dump`, el diccionario de datos por tabla, la lista de migraciones y las firmas de las 185 funciones se encuentran en el archivo de anexos.

AVANCE DEL PROYECTO.

Este avance reúne lo que existe hoy en el repositorio del proyecto (rama principal) y el trabajo nuevo de la estación de aula. Cada cifra procede de una consulta o de una ejecución hecha para este reporte, y se indica qué no se ha verificado.

1. Estado por parte.

Parte	Materia	Qué está hecho	Verificación
Base de datos	Programación Avanzada	PostgreSQL con 20 tablas de datos, 135 columnas, restricciones, disparadores, funciones y políticas de seguridad por fila	Esquema comparado con la base en ejecución; pruebas SQL del proyecto: 37 archivos, 1508 comprobaciones positivas y 953 negativas, 0 fallos
Formularios web	Programación Avanzada	41 formularios y flujos guiados (acceso, alumnos, grupos, catálogos, justificantes, registro facial) con 114 capturas de pantalla	Pruebas automáticas de formularios: 2008 pruebas, 2007 pasan y 1 falla por un temporizado del detector de rostros; no se corrió la suite completa, ni el análisis de código ni la compilación de producción
Servicio de reconocimiento	Programación Avanzada	Servicio con índice FAISS y registro facial con consentimiento	No se ejecutó la verificación de precisión en este avance
Estación de aula (Arduino)	Sistemas Digitales	Circuito, firmware, protocolo serie y programa en Node del puerto serie, escritos en este avance	Compila para Arduino UNO sin errores ni avisos y se ejecutó en un simulador de instrucciones; pruebas de lógica en el equipo (65 y 38 casos, 0 fallidos). **No se ha probado con hardware físico**
Integración estación y SUSAN	Ambas	Protocolo serie definido; el programa en Node responde a la estación con una sesión simulada	Pendiente: tabla de tarjetas y registro de asistencia en la base de datos

Nota. Elaboración propia. La verificación de cada requerimiento de la especificación (estado Implementado, Parcial o Pendiente) no se completó en este avance.

2. Integración de las materias.

- **Sistemas Digitales:** la estación de aula, con sus entradas y salidas digitales, los buses SPI e I2C, el antirrebote del pulsador y la máquina de estados del firmware.
- **Programación Avanzada:** la aplicación web y sus formularios, la base de datos, el servicio de reconocimiento y el programa que maneja el puerto serie en la computadora.

3. Próximos pasos.

1. Armar la estación con los componentes de la lista de materiales y probar el cableado, la lectura de tarjetas, la dirección de la pantalla y el antirrebote reales.
2. Crear en la base de datos la tabla que asocia cada tarjeta con un alumno y una función que registre la asistencia desde la estación.
3. Conectar el programa en Node con esa función y probar el flujo completo con una sesión abierta.
4. Completar la verificación de cada requerimiento funcional, no funcional y regla de negocio de la especificación.
5. Correr la suite completa de pruebas, el análisis de código y la compilación de producción de la aplicación web.
6. Medir la precisión del reconocimiento y el ahorro de tiempo en un piloto con un grupo.

VIABILIDAD TÉCNICA Y RIESGOS.

La propuesta es viable con material de un laboratorio de sistemas digitales y con software gratuito o libre. La aplicación web, el servicio de reconocimiento y la base de datos ya se ejecutan y se prueban en el repositorio del proyecto. La estación de aula con Arduino es trabajo nuevo de este avance y se diseñó y verificó solo en software: la lógica de su protocolo se probó en el equipo de desarrollo y el firmware se compila para el microcontrolador del UNO, pero la estación no se ha probado en hardware físico. Por eso el riesgo principal no es de software, sino la disponibilidad del material y la validación del circuito armado, junto con la calibración del reconocimiento facial con datos reales, que sigue pendiente.

1. Materiales y software.

Elemento	Uso	Disponibilidad
Arduino UNO R3	Microcontrolador de la estación de aula (ATmega328P, 5 V)	Por confirmar con el laboratorio
Módulo lector MFRC522	Lectura del UID de la tarjeta por SPI (lógica de 3.3 V)	Por confirmar
Tarjetas o llaveros RFID	Credencial personal del alumno sin consentimiento facial	Por confirmar
Pantalla LCD 16x2 con módulo I2C	Muestra la sesión, la última lectura y el estado de la estación	Por confirmar
LEDs, resistencias, zumbador, pulsador y protoboard	Indicadores, divisores de tensión, aviso sonoro y botón	Por confirmar
Cable USB	Alimentación y enlace serie con el equipo del docente	Por confirmar
Computadora del docente	Navegador, aplicación web y programa en Node del puerto serie	Equipo del laboratorio o de un integrante
Arduino IDE o arduino-cli	Compilar y cargar el firmware en el UNO	Software libre
Librerías MFRC522 y LiquidCrystal I2C	Control del lector y de la pantalla desde el firmware	Descarga gratuita
avr-gcc, avr-libc y g++	Compilar el firmware sin el IDE y probar la lógica del protocolo en el PC	Software libre
Node.js 20.9 o superior	Ejecuta la aplicación web y el programa del puerto serie	Software libre
Paquete `serialport` de npm	Acceso al puerto serie desde Node	Software libre
Next.js 16.3.8, React 19.2.8 y Tailwind CSS 4	Aplicación web (App Router)	Software libre
Supabase (`@supabase/supabase-js` ^2.117.2)	Autenticación, almacenamiento y API sobre PostgreSQL	Modo local sin cuenta; en la nube, cuenta por confirmar
PostgreSQL 16 (entorno local de pruebas; la configuración de Supabase declara la versión 17)	Base de datos: 20 tablas, funciones RPC, RLS y triggers	Software libre

Elemento	Uso	Disponibilidad
Python 3.12, FastAPI y Uvicorn	Servicio de reconocimiento facial	Software libre
FAISS (`faiss-cpu` 1.9 o superior)	Índice de búsqueda por similitud de vectores de 512 dimensiones	Software libre
InsightFace (modelo `buffalo_s`) y ONNX Runtime	Detección y vectorización de rostros (ArcFace, 512 dimensiones)	Descarga gratuita; modelos solo para uso no comercial
Vitest y pytest	Pruebas de la aplicación web y del servicio de reconocimiento	Software libre
Playwright con Chromium	Capturas automáticas de pantallas y diagramas de este reporte	Software libre

Nota. Las versiones del software provienen de `web/package.json`, `recognition-service/requirements.txt` y los scripts del repositorio. Las características eléctricas del lector se toman de NXP Semiconductors (2016) y las de la placa y de los buses SPI e I2C (Wire), de Arduino (s. f.-a, s. f.-c, s. f.-d). La disponibilidad del material se confirmará con el laboratorio.

2. Riesgos.

Riesgo	Efecto	Mitigación
La estación no se ha probado en hardware físico	Pueden aparecer diferencias entre lo simulado y lo real: cableado, tiempos de la librería del lector y de la pantalla, zumbador	Se entrega validada en software; el armado y la prueba en el laboratorio quedan como paso siguiente. Sin la estación, el pase manual y la cámara siguen funcionando
El MFRC522 trabaja a 3.3 V y el UNO a 5 V	Una señal de 5 V sin reducir puede dañar el módulo	Divisores resistivos en las líneas que salen del UNO hacia el módulo; alimentar el módulo desde el pin de 3.3 V y revisar el cableado antes de energizar (NXP Semiconductors, 2016)
El UID de una tarjeta puede copiarse y el sistema no detecta si una foto sustituye al rostro	Un alumno podría registrar a otro con una tarjeta copiada o prestada, o con una fotografía	La tarjeta es un respaldo de conveniencia, no autenticación fuerte. Lo atenúan la supervisión del docente, la corrección de registros (RN-08), las 3 lecturas en capturas distintas y la cola de revisión. La detección de vida queda como riesgo abierto

Riesgo	Efecto	Mitigación
El UID aún no se asocia a un alumno en la base de datos	La estación lee y envía el UID, pero todavía no se asienta asistencia con ella	Siguientes pasos: tabla de credenciales UID-alumno y una función RPC con el mismo patrón del esquema (RLS y bitácora). La estación nunca cierra sesiones
Permisos, controladores y puerto serie ocupado	Node no abre el puerto y la estación queda sin enlace con la aplicación	Dar al usuario permiso sobre el dispositivo serie, instalar el controlador si la placa lo requiere y cerrar el monitor serie del IDE. Tras 30 s sin tráfico válido, el firmware lo indica con el LED ámbar
Precisión del reconocimiento y condiciones de luz	Un umbral exigente llena la cola de revisión; uno laxo causa atribuciones equivocadas	Umbral (0.700 desde el 2026-10-03), mínimo de 3 lecturas e intervalo de 15 s son parámetros calibrables. Las mediciones usan variantes de una foto por persona, no fotos independientes; la calibración de campo en un piloto de 1 o 2 grupos está pendiente
Tratamiento del rostro como dato sensible y revisión jurídica pendiente	Reconocer sin consentimiento vigente o filtrar datos biométricos	Ningún vector ni imagen en tablas relacionales (RNF-03); sin consentimiento hay pase manual o tarjeta RFID (RN-13, RNF-06); objetos solo por URL firmada de hasta 300 s (Supabase, s. f.-b). El aviso y la vigencia del consentimiento los debe aprobar el área jurídica de la UNACH (INAI, 2018; LGPDPPSO, 2025; LPDPPSOCH, 2025)
Disponibilidad de la cámara del aula	Sin cámara no hay reconocimiento automático en esa sesión	Cámara de tipo `navegador` (webcam del equipo del docente), prevista para un piloto sin cámaras IP; respaldo con pase manual (RF-09) y con la estación RFID
Presupuesto de rendimiento de la interfaz	Carga inicial más lenta de lo previsto en algunas pantallas	`/docente` quedó en 226.1 KB comprimidos (+8.0 %), dentro del límite de +15 %. `/alumno`, `/reportes` y `/docente/sesiones/[id]` lo superan y no tienen presupuesto propio; falta repetir la medición del tiempo de ejecución con `next start`
El reloj y la cámara de demostración solo existen en modo local	Los resultados de la demostración (hora fija, rostros sintéticos) no prueban la precisión real	Se activan solo con `SUSAN_LOCAL=1` y el reloj simulado habilitado en la base, nunca en Supabase real. Con rostros reales se usa el modo `insightface`; la precisión se valida en el piloto

Nota. Los riesgos del reconocimiento, la privacidad y el rendimiento provienen de `docs/DECISIONS.md`, de `docs/DESIGN_PROGRESS.md` y de la especificación del primer avance. Los de la estación son de diseño y no se han observado en hardware. Los modelos preentrenados de InsightFace son de uso no comercial, algo compatible con un proyecto académico pero que debe revisarse si el sistema se desplegara fuera de ese ámbito.

REFERENCIAS BIBLIOGRÁFICAS.

Arduino. (s. f.-a). *Arduino Uno Rev3*. https://docs.arduino.cc/hardware/uno-rev3/

Arduino. (s. f.-b). *Serial*. https://www.arduino.cc/reference/en/language/functions/communication/serial/

Arduino. (s. f.-c). *SPI*. https://www.arduino.cc/reference/en/language/functions/communication/spi/

Arduino. (s. f.-d). *Wire*. https://www.arduino.cc/reference/en/language/functions/communication/wire/

Deng, J., Guo, J., Xue, N., y Zafeiriou, S. (2019). ArcFace: Additive angular margin loss for deep face recognition. En *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 4685–4694). IEEE. https://doi.org/10.1109/CVPR.2019.00482

Instituto Nacional de Transparencia, Acceso a la Información y Protección de Datos Personales. (2018). *Guía para el tratamiento de datos biométricos*. https://home.inai.org.mx/wp-content/documentos/DocumentosSectorPublico/GuiaDatosBiometricos_Web_Links.pdf

Johnson, J., Douze, M., y Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data, 7*(3), 535–547. https://doi.org/10.1109/TBDATA.2019.2921572

Ley de Protección de Datos Personales en Posesión de Sujetos Obligados del Estado de Chiapas. (2025, 18 de junio). *Periódico Oficial del Estado de Chiapas*, núm. 045 (Decreto número 276). Última reforma publicada el 17 de junio de 2026. https://www.congresochiapas.gob.mx/new/Info-Parlamentaria/LEY_0135.pdf

Ley General de Protección de Datos Personales en Posesión de Sujetos Obligados. (2025, 20 de marzo). *Diario Oficial de la Federación*. Última reforma publicada el 14 de noviembre de 2025. https://www.diputados.gob.mx/LeyesBiblio/pdf/LGPDPPSO.pdf

NXP Semiconductors. (2016). *MFRC522: Standard performance MIFARE and NTAG frontend* (Rev. 3.9) [Hoja de datos]. https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf

PostgreSQL Global Development Group. (s. f.). *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/

Supabase. (s. f.-a). *Row Level Security*. https://supabase.com/docs/guides/database/postgres/row-level-security

Supabase. (s. f.-b). *Serving assets from Storage*. Recuperado el 14 de septiembre de 2026, de https://supabase.com/docs/guides/storage/serving/downloads

Vercel. (s. f.). *Next.js documentation*. https://nextjs.org/docs